Context Mixing from First Principles

A Deep, Example-Driven Guide for Aspiring Hutter Prize Competitors

Compiled October 2026

Preface

This document is written for one purpose: to take you from "I have heard that the best compressors use *context mixing*" to a state where you can *design* a context-mixing compressor yourself, and understand every number it produces. It is deliberately long and deliberately concrete. Almost every idea is followed by a worked example with real numbers, and the last appendix contains a complete, working (about 200 lines of C) context-mixing compressor that I compiled and verified to round-trip, together with an ablation study showing what each component is worth.

How to read it. Part I builds the mathematical foundations (information theory and arithmetic coding). Part II builds a context-mixing compressor component by component: contexts, counters, bit histories, the logistic mixer, SSE/APM stages and the match model. Part III moves to what is specific to the Hutter Prize: text and XML modelling for `enwik9`, dictionary preprocessing, neural components (the LSTM used in `cmix`), engineering under the time and memory limits, and a methodology for developing your own algorithm. The appendices contain the reference implementation, notation, hints and a reading list.

What is verified and what is not. Three kinds of statements appear in this document, and I try to keep them apart:

- **Mathematics** (derivations, worked examples). I recomputed every numeric example by program; they are exact up to the printed rounding.
- **Experiments.** The tables in Appendix A were produced by running the included code on a 1 MB mixed English text sample from the machine where this document was prepared. This sample is *not* Wikipedia, it contains a lot of near-duplicate license text, and so absolute numbers (especially the benefit of the match model) are *much more favourable* than you will see on `enwik9`. Use them to learn the *method* of ablation, not as predictions.
- **Facts about the Prize and about other people's programs** (current record, rules, what `cmix`/`phda9`/STARLIT do). The record and rule figures were checked against the prize web pages in October 2026, but rules and records change; always re-read `http://prize.hutter1.net/`. Statements about the internals of specific third-party programs are from my background knowledge and the sources cited; where I am not sure I say so. **Before you copy any design decision, read the actual source code** (Appendix E lists what to read).

Prerequisites. Comfortable programming (C/C++ is the lingua franca of this field), basic probability, logarithms, and partial derivatives. Everything else is introduced.

Contents

Preface ii

I Foundations 1

1 The Hutter Prize and the Big Picture 2

1.1 What the prize actually asks for . . . 2
1.2 The state of the art . . . 2
1.3 The compute budget, quantified . . . 3
1.4 Compression is prediction . . . 3
1.5 A map of the compressor landscape . . . 3
1.6 Roadmap . . . 4

2 Information Theory You Actually Need 5

2.1 Surprise, bits and code length . . . 5
2.2 Entropy . . . 5
2.3 Cross-entropy: the real cost of an imperfect model . . . 6
2.4 Mixtures of models and the cost of "not knowing which is right" . . . 7
2.5 Kolmogorov complexity, MDL and Hutter's motivation . . . 7
2.6 How performance is measured . . . 7

3 Arithmetic Coding 9

3.1 The idea: nested intervals . . . 9
3.2 Integer implementation (what you will actually write) . . . 10
3.3 Precision and its cost . . . 10
3.4 Alternatives . . . 11

II Building a Context-Mixing Compressor 12

4 Binary Decomposition and Contexts 13

4.1 Why predict one bit at a time? . . . 13
4.2 The partial-byte context c_0 . . . 13
4.3 What is a context? . . . 14
4.4 PPM versus context mixing . . . 15
4.5 Hashing and memory layout . . . 15

5 Predicting from One Context: Counters and Bit Histories **16**
5.1 Plain frequency counts and the KT estimator . . . 16
5.2 Adaptive-rate probability counters . . . 16
5.3 Bit histories and non-stationary state machines . . . 17
5.4 Indirect modelling: learning what a history means . . . 17
5.5 Beyond stretch(p): richer inputs per model . . . 18

6 The Heart of Context Mixing: Logistic Mixing **19**
6.1 The problem . . . 19
6.2 Stretch and squash . . . 19
6.3 Logistic mixing . . . 20
6.3.1 Why in the logistic domain? The product-of-experts view . . . 20
6.4 Learning the weights online . . . 21
6.5 A complete numerical example . . . 21
6.6 Fixed-point implementation . . . 22
6.7 Weight sets selected by context . . . 22
6.8 Two-layer mixing networks . . . 23
6.9 Ablation results from the reference implementation . . . 23

7 SSE / APM: Fixing Calibration After Mixing **25**
7.1 Idea . . . 25
7.2 Chaining and averaging . . . 26

8 The Match Model and Long-Range Structure **27**
8.1 Why a separate model for long repeats . . . 27
8.2 Algorithm . . . 27
8.3 Making the match model smarter . . . 28
8.4 Long-range structure in Wikipedia . . . 29

9 Putting It Together: One Bit, Start to Finish **30**
9.1 The full loop . . . 30
9.2 A numerical walk-through of a single bit . . . 30

9.3 What dominates the time? . . . 31

III Toward the Hutter Prize **32**

10 Modelling Wikipedia: Text, XML and Preprocessing **33**

10.1 What is in `enwik9`? . . . 33
10.2 Preprocessing: transform the data before modelling . . . 33
10.3 Article reordering . . . 34
10.4 Specialised context models for Wikipedia . . . 34
10.5 Bytes, words or tokens? . . . 35
10.6 Tuning on `enwik8` versus `enwik9` . . . 35

11 Neural Components: the LSTM and `cmix` **36**

11.1 cmix in one paragraph . . . 36
11.2 The LSTM predictor . . . 36
11.3 From a byte distribution to bit probabilities . . . 37
11.4 Online training . . . 37
11.5 Budgeting compute for a neural model . . . 38
11.6 Other neural ideas . . . 38
11.7 Floating-point determinism . . . 38

12 Engineering Under the Prize Limits **39**

12.1 The constraints, restated as design requirements . . . 39
12.2 Speed engineering . . . 39
12.3 Memory engineering . . . 39
12.4 Correctness engineering . . . 40
12.5 Development workflow . . . 40
12.6 Parameter tuning . . . 40
12.7 Shrinking the decompressor . . . 41

13 Designing Your Own Algorithm **42**

13.1 An honest assessment . . . 42
13.2 Staged plan . . . 42
13.3 A checklist for evaluating every idea . . . 42
13.4 Where gains may still be hiding . . . 43
13.5 Social and practical advice . . . 43

A A Complete Teaching Compressor (tinycm) **44**

A.1 Architecture 44
A.2 Results on a 1 MB sample 44
A.3 Source code 44
A.4 Suggested modifications (a research-training programme) 48

B Notation **49**

C Glossary **50**

D Suggested Reading and Code to Study **51**

Part I

Foundations

Chapter 1

The Hutter Prize and the Big Picture

1.1 What the prize actually asks for

The Hutter Prize (officially the "500 000 € Prize for Compressing Human Knowledge") rewards lossless compression of `enwik9`, the first 10^9 bytes of an XML dump of the English Wikipedia taken in March 2006. The prize was announced in 2006; in February 2020 both the data (from the 100 MB `enwik8` to the 1 GB `enwik9`) and the prize pool (from 50 000 to 500 000 euros) were multiplied by ten.

The scoring rule

You submit a compressor `comp9.exe` that turns `enwik9` into an archive `archive9.exe`, which is a *self-extracting* program that reproduces `enwik9` exactly. Your score is the total

$$S \;=\; \text{size}(\texttt{comp9.exe}\text{ as a zip}) \;+\; \text{size}(\texttt{archive9.exe}),$$

and if L is the previous record and $Z = 500\,000$€, the award is $Z\,(L - S)/L$. The smallest claimable prize is 5 000€, i.e. you must beat the previous record by at least 1%, and there is a 30-day public-comment period before payment.

The important consequence of S including the decompressor is that you cannot hide information in a huge pretrained model: every byte of dictionary, every learned parameter you ship counts against you, exactly like compressed data. This is the idea of the *minimum description length* principle in action (Chapter 2).

Resource limits. At the time of writing the official page lists, for recent records, about 10 GB of RAM and about 50 hours of run time on a single core (and, per the rules mirrored in public README files, on the order of 100 GB of disk). The exact limits and the exact reference hardware are set by the prize committee and "may change without notice"; check the rules page before you design anything around them.

1.2 The state of the art

Table 1.1 lists the `enwik9` records as published on the prize page and in public mirrors of it.

- `gzip -9` turns `enwik9` into roughly 323 MB, i.e. $\approx$ 2.58 bits per byte. The record is $110\,793\,128 \cdot 8/10^9 \approx 0.886$ bits per byte: about 2.9× smaller than gzip.

Author	Date	S (bytes)	bits/byte	Program
A. Rhatushnyak	2019	116 673 681	0.934	phda9 v1.8 (pre-prize for enwik9)
A. Margaritov	31 May 2021	115 352 938	0.923	STARLIT + cmix
S. Kumar	16 Jul 2023	114 156 155	0.913	cmix-based
K. Orav	2 Feb 2024	112 578 322	0.900	fx-cmix
K. Orav & B. Knoll	3 Sep 2024	110 793 128	0.886	fx2-cmix

Table 1.1: Recent `enwik9` records (sizes include the decompressor). The last row is the record I could find as of October 2026; verify on the official page.

- Going from 116.67 MB (2019) to 110.79 MB (2024) is a total improvement of only 5.04% in five years. Every further 1% is ≈ 1.1 MB, and that is the minimum step to win money: to earn a prize one must reach $S \le 0.99 \cdot 110\,793\,128 \approx 109\,685\,196$ bytes.
- This is a mature race. You will not win by a clever trick on top of gzip; you must understand and then *extend* the state of the art, which today is a context-mixing ensemble with a neural network, tightly tuned for this exact file.

1.3 The compute budget, quantified

Because decompression must replay the model exactly, a context-mixing compressor spends the *same* time to decode as to encode. So the 50-hour limit applies to the model itself:

$$\frac{50 \text{ h} \cdot 3600 \text{ s/h}}{10^9 \text{ bytes}} = 180 \ \mu\text{s per byte} \ = \ 22.5 \ \mu\text{s per bit} \ \approx \ 5.6 \text{ KB/s}.$$

For comparison a plain `lpaq`-style compressor processes on the order of a megabyte per second, so the prize allows more than two orders of magnitude more computation per byte than a fast CM. This is why the winning designs use hundreds of models and an LSTM: the budget is spent on *more modelling*, not on entropy coding.

1.4 Compression is prediction

A deep fact, used throughout: **an adaptive model that assigns probability p_t to the symbol that actually occurs at time t can be turned, by arithmetic coding, into a compressor whose output length is $\sum_t -\log_2 p_t$ bits (plus a couple of bits).** The entropy coder is a solved, cheap, nearly lossless component. All the difficulty is in *prediction*: the quality of the probabilities. This is why the prize is considered a test of "intelligence": a model that predicts Wikipedia text well must have absorbed syntax, semantics and facts. The prize FAQ states this motivation explicitly.

1.5 A map of the compressor landscape

Dictionary methods (LZ77, LZ78, LZMA). Replace repeats with pointers. Fast, but only exploit exact repeats and code lengths are coarse.

Symbol-ranking, BWT (bzip2). Reorder the data to make it locally homogeneous, then code with simple adaptive models.

PPM (Prediction by Partial Matching). Predict with the longest context seen so far; if the symbol is new, "escape" to a shorter context. One context at a time is consulted.

Context mixing (PAQ family, `cmix`). Consult *many* context models at once, each giving a probability for the next *bit*; combine them with an adaptive, learned (online-trained) logistic function; refine with SSE; arithmetic-code the result. This is the subject of this document.

Neural predictors (LSTM, Transformer). A single learned network predicts the next symbol directly. Used as one more input to the mixer in `cmix`.

On the 100 MB `enwik8` the first Hutter Prize baseline (Mahoney's `paq8f`, 2006) was 18 324 887 bytes, and the first prize-winning entry (Rhatushnyak's `paq8hp5`, September 2006) reached 17 073 018 bytes. The lineage `paq8f` → `paq8hp` → `phda9` → `cmix` → `fx-cmix` is a straight line of context-mixing development.

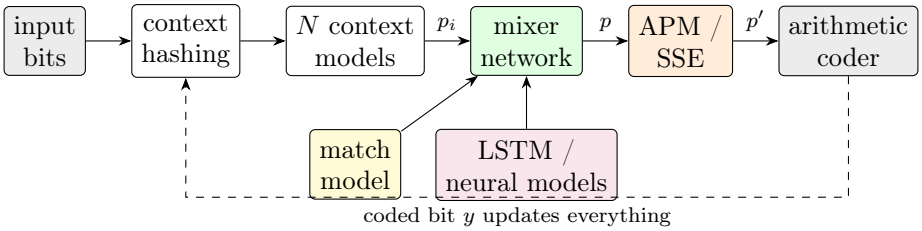


Figure 1.1: The bit-level loop of a context-mixing compressor. After each bit is coded, *every* component is updated with the true bit; the decoder runs the identical loop.

1.6 Roadmap

Chapters 2–3 give the theory of coding. Chapters 4–9 build a complete CM compressor. Chapters 10–12 specialise to Wikipedia text, neural components and engineering. Chapter 13 is a development methodology. If you are impatient, read Chapter 6 (mixing) and Appendix A (the code) first, then return.

Chapter 2

Information Theory You Actually Need

2.1 Surprise, bits and code length

If an event has probability p, observing it conveys $-\log_2 p$ bits of *surprise*. A compressor that assigns probability p to what actually happens is, in effect, "paying" $-\log_2 p$ bits for it.

probability p of what happened	cost $-\log_2 p$ (bits)	comment
1/2	1.000	fair coin
0.9	0.152	expected event
0.99	0.0145	near-certain
0.999	0.00144	
0.1	3.322	surprising
0.01	6.644	very surprising
0.001	9.966	

Table 2.1: Cost of an outcome as a function of the probability the model gave it.

Overconfidence is expensive

If the model says $P(\text{bit} = 1) = 0.999$ and the bit is 0, the cost is $-\log_2 0.001 = 9.97$ bits. Each *correct* prediction at that confidence costs only 0.0014 bits, so a single miss wipes out the cost of about 6 900 correct predictions. The lesson, repeated throughout this document: **calibration** (stated probability = actual frequency) matters as much as accuracy. SSE/APM stages (Chapter 7) exist precisely to fix calibration.

2.2 Entropy

For a source with symbol probabilities $p(x)$ the *entropy* is

$$H(X) = -\sum_{x} p(x) \log_2 p(x) \quad \text{bits per symbol.}$$

Shannon's source coding theorem: no lossless code can have average length below H, and codes exist that get within one bit (arithmetic coding gets within a fraction of a bit for the *whole message*).

$P(1)$	$H_b(P(1))$ bits
0.5	1.0000
0.9	0.4690
0.99	0.0808
0.999	0.0114

Table 2.2: Binary entropy. A bit that is 99% predictable carries only 0.08 bits of information.

Example 2.1 — Entropy of "abracadabra"

The string `abracadabra` has 11 symbols: `a`×5, `b`×2, `r`×2, `c`×1, `d`×1.

- **Order-0 model** (frequencies only): $H_0 = -\frac{5}{11}\log_2\frac{5}{11} - 2\cdot\frac{2}{11}\log_2\frac{2}{11} - 2\cdot\frac{1}{11}\log_2\frac{1}{11} = 2.0404$ bits/symbol, so $11 \times 2.0404 = 22.44$ bits, versus 88 bits as 8-bit ASCII.
- **Order-1 model** (condition on the previous letter). The ten transitions are `ab`×2, `br`×2, `ra`×2, `ac`, `ca`, `ad`, `da`. After `b`, `r`, `c` and `d` the next letter is *always the same*, so those six transitions cost 0 bits. Only the context `a` is uncertain: it is followed by `b` twice, and by `c` and `d` once each, so the code lengths are $2\cdot 1 + 1\cdot 2 + 1\cdot 2 = 6$ bits. The ideal order-1 cost of the ten transitions is therefore exactly 6.0 bits, against $10 \times 2.04 = 20.4$ bits for order 0.

The lesson: richer context ⇒ lower entropy *if the statistics are known in advance*. A real compressor must *learn* them while coding, which costs extra; that is the whole subject of adaptive modelling.

2.3 Cross-entropy: the real cost of an imperfect model

Let the data be generated by the true distribution p but coded with a model q. The expected code length per symbol is the *cross-entropy*

$$H(p, q) = -\sum_x p(x)\log_2 q(x) = H(p) + D_{\mathrm{KL}}(p\,\|\,q), \qquad D_{\mathrm{KL}}(p\|q) = \sum_x p(x)\log_2\frac{p(x)}{q(x)} \geq 0.$$

The compressed size of a particular sequence $x_1 \ldots x_n$ under a sequential model is exactly

$$\boxed{\ell(x_{1:n}) = \sum_{t=1}^{n} -\log_2 q(x_t \mid x_{1:t-1})}$$

This is the *only quantity you ever optimise*. Every technique in this document is a way of making this sum smaller on one specific file.

Example 2.2 — Chain rule and sequential prediction

Any joint distribution factorises as $q(x_{1:n}) = \prod_t q(x_t \mid x_{<t})$. So building a compressor is equivalent to building a *next-symbol predictor*. The arithmetic coder never needs to know anything else about the model.

2.4 Mixtures of models and the cost of "not knowing which is right"

Suppose you have K candidate models $q_1, \ldots, q_K$ and no idea which is best on this file. A *Bayesian mixture* $q_{\text{mix}}(x_{1:n}) = \sum_k w_k q_k(x_{1:n})$ with prior weights w_k (summing to 1) satisfies

$$-\log_2 q_{\text{mix}}(x_{1:n}) \;\le\; -\log_2 q_k(x_{1:n}) + \log_2 \frac{1}{w_k} \quad \text{for every } k.$$

Proof. $q_{\text{mix}} \ge w_k q_k$ since all terms are non-negative; take $-\log_2$. □

So a mixture is never worse than the best model plus $\log_2(1/w_k)$ bits: with K equal weights, a one-time cost of $\log_2 K$ bits for "choosing" the best model, on the *whole file*. This is an astonishingly good guarantee, and it is the theoretical reason to ensemble many models: with $K = 1000$ models you pay only ≈ 10 bits in total for not knowing which one is best.

Key idea

Real context mixing improves on the Bayesian mixture in two ways: (i) the weights are learned *continuously* and may depend on local context (the best model for XML tags differs from the best model for prose), and (ii) the combination is done in the logistic domain, which gives sharper predictions when models agree (Chapter 6). The price is that the simple $\log_2 K$ guarantee no longer applies; the benefit is that the compression is far better in practice.

2.5 Kolmogorov complexity, MDL and Hutter's motivation

The Kolmogorov complexity $K(x)$ is the length of the shortest program that prints x. It is uncomputable, but any compressor gives an upper bound: $K(x) \le |\text{decompressor}| + |\text{archive}|$. The Hutter Prize's S is exactly this upper bound for `enwik9`, with the decompressor included. Hutter's universal-AI theory (the *AIXI* agent) is built on Solomonoff induction, which predicts with a mixture over *all* computable models weighted by $2^{-\text{length}}$; context mixing is a practical, drastically restricted cousin of that idea. Whether or not you care about AGI, the take-away is practical: **model size is part of the score**, so a better model is only worth it if it saves more bytes than it costs to describe.

2.6 How performance is measured

- **bits per character** bpc $= 8\,|\text{archive}|/|\text{original}|$. On text $\approx$ 1.6–2 for good conventional compressors, ≈ 0.9 for Hutter Prize records on `enwik9`.
- **compression ratio** $= |\text{original}|/|\text{archive}|$ (the record has ratio about 9.03).
- For the Prize: the total S in bytes, nothing else (subject to the resource limits).

Exercise 2.1 — Cost of a streak

A model predicts $P(1) = 0.99$ for 1000 bits, and 10 of them turn out to be zeros. Compute the total code length in bits and compare with always predicting 0.5. *Answer:* $990 \times 0.0145 + 10 \times 6.644 = 14.4 + 66.4 = 80.8$ bits versus 1000 bits.

Exercise 2.2 — Mixture bound

Two models give a sequence probabilities 2^{-1000} and 2^{-1100}. With equal prior weights, what does the mixture cost? *Answer:* at most $1000 + 1 = 1001$ bits.

Chapter 3

Arithmetic Coding

3.1 The idea: nested intervals

Represent the whole message as a number in $[0, 1)$. Start with the interval $[0, 1)$. For every bit, split the current interval in proportion to the model's probabilities and keep the sub-interval corresponding to the bit that actually occurred. After all bits, any number inside the final interval identifies the message; a number needs about $-\log_2(\text{width})$ bits to specify.

Convention used here and in the code: bit 1 takes the *lower* part of width $p_1 \cdot (\text{width})$, bit 0 takes the upper part.

Example 3.1 — Encoding the bits 1 0 1 1

Model probabilities $P(1) = 0.8, 0.6, 0.7, 0.9$ for the four bits.

step	$P(1)$	bit y	split point	new interval	width
0				$[0, 1)$	1
1	0.8	1	$0 + 1 \cdot 0.8 = 0.8$	$[0, 0.8)$	0.8
2	0.6	0	$0 + 0.8 \cdot 0.6 = 0.48$	$[0.48, 0.8)$	0.32
3	0.7	1	$0.48 + 0.32 \cdot 0.7 = 0.704$	$[0.48, 0.704)$	0.224
4	0.9	1	$0.48 + 0.224 \cdot 0.9 = 0.6816$	$[0.48, 0.6816)$	0.2016

Final width 0.2016, and indeed $-\log_2 0.2016 = 2.3104 = 0.3219 + 0.7370 + 0.5146 + 0.1520$, the sum of the four individual costs. Any number in $[0.48, 0.6816)$ works; $0.5 = 0.1_2$ lies inside, so *one bit* ("1") suffices here (luck: the guaranteed bound is $\lceil -\log_2 w \rceil + 1 = 4$ bits). **Decoding the number** 0.5, knowing the same probabilities:

step	$P(1)$	current value v	test	decoded bit, rescaled v
1	0.8	0.5	$v < 0.8$	1; $v \leftarrow 0.5/0.8 = 0.625$
2	0.6	0.625	$v \geq 0.6$	0; $v \leftarrow (0.625 - 0.6)/0.4 = 0.0625$
3	0.7	0.0625	$v < 0.7$	1; $v \leftarrow 0.0625/0.7 = 0.0893$
4	0.9	0.0893	$v < 0.9$	1

The decoder recovers 1011. Note that **the decoder must compute exactly the same probabilities as the encoder**, bit for bit. This symmetry is why a CM decompressor costs as much time as the compressor, and why floating-point determinism matters (Chapter 11).

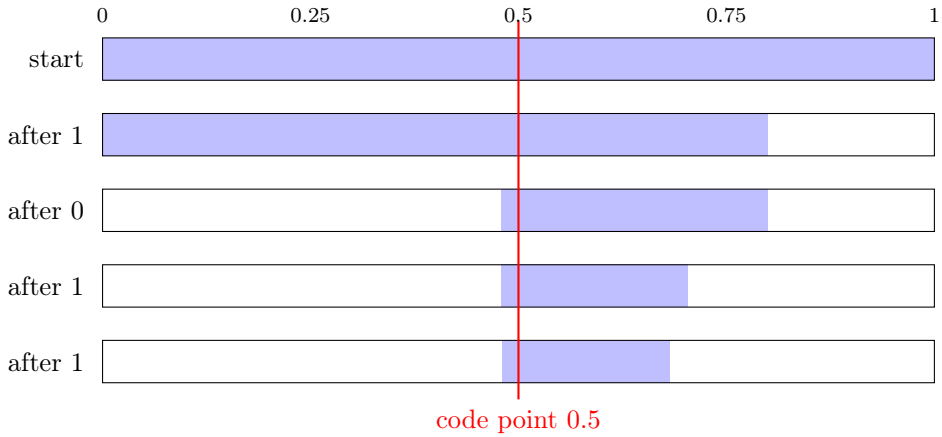


Figure 3.1: The interval shrinks with every coded bit; its final width equals the product of the probabilities of the actual bits.

3.2 Integer implementation (what you will actually write)

Floating point is not used. The lpaq/paq8 coder keeps a 32-bit interval $[x_1, x_2]$ and a 12-bit probability $p \in [1, 4095]$ that the next bit is 1:

```
xmid = x1 + ((x2 - x1) >> 12) * p        // split point
if (y == 1) x2 = xmid else x1 = xmid + 1  // keep the sub-interval
while (((x1 ^ x2) & 0xff000000) == 0) {   // top bytes equal: they are final
    output_byte(x2 >> 24)
    x1 = x1 << 8;  x2 = (x2 << 8) | 255
}
```

The loop emits a byte as soon as the leading 8 bits of both ends agree, because no future bit can change them. At the end four bytes of x_1 are flushed. The decoder keeps the same $[x_1, x_2]$, reads the code bytes into x, and decodes $y = (x \le x_{mid})$.

Example 3.2 — Integer coding with 12-bit probabilities

Start with $x_1 =$ `0x00000000`, $x_2 =$ `0xFFFFFFFF` and $p = 3277$ ($\approx 0.8 \cdot 4096$).

1. $(x_2 - x_1) \gg 12 =$ `0xFFFFF` $= 1\,048\,575$; $x_{mid} = 1\,048\,575 \cdot 3277 =$ `0xCCCFF333`. The bit is $y = 1$, so $x_2 \leftarrow$ `0xCCCFF333`. The top bytes are `00` $\neq$ `CC`: no output.
2. $p = 2458$ ($\approx 0.6 \cdot 4096$): $(x_2 - x_1) \gg 12 =$ `0xCCCFF`, $x_{mid} =$ `0x7AE84866`. The bit is $y = 0$, so $x_1 \leftarrow x_{mid} + 1 =$ `0x7AE84867`. Top bytes `7A` $\neq$ `CC`.

As real numbers: `0x7AE84867`$/2^{32} = 0.4801$ and `0xCCCFF333`$/2^{32} = 0.8000$, matching the interval $[0.48, 0.8)$ of the previous example.

3.3 Precision and its cost

Two sources of inefficiency:

1. **Truncation** of $(x_2 - x_1) \gg 12$ and of p to 12 bits. The interval is never less than 2^{24} wide after normalisation, so the relative error is tiny ($\lesssim 2^{-12}$ in the worst case).
2. **Probability resolution.** With $p \le \frac{4095}{4096}$ the cheapest possible bit costs about 0.00035 bits, i.e. 0.0028 bits per byte. For ordinary text (0.9 bits per byte) that is irrelevant. For highly redundant data (e.g. long exact repeats) it is a measurable floor, which is why newer

PAQ versions use 16-bit probabilities in the final stages. For `enwik9` the effect is small but non-zero; it is one of the many places where 0.1% is hiding.

Probability 0 and 1

Never pass $p = 0$ or $p = 4096$ to the coder. A zero-probability event that occurs makes the interval empty and breaks decoding. Always clamp to $[1, 4095]$ (or the equivalent in your precision).

3.4 Alternatives

Range coders are arithmetic coders working on bytes with multiplications on larger alphabets. Equivalent in efficiency.

ANS (asymmetric numeral systems) gives arithmetic-coding efficiency with table-driven speed. They are excellent for *static* or slowly changing distributions (zstd, JPEG XL). They decode in reverse order of encoding, which is awkward when the probabilities depend on the data decoded so far, as in CM. For the Prize the coder is not the bottleneck; the binary arithmetic coder above is the standard choice.

Exercise 3.1

Encode the bits 0 0 1 with $P(1) = 0.3, 0.2, 0.5$ using the convention of this chapter and give the final interval and its width. *Answer:* step 1 ($y = 0$): split at 0.3, interval $[0.3, 1)$; step 2 ($y = 0$): split at $0.3 + 0.7 \cdot 0.2 = 0.44$, interval $[0.44, 1)$; step 3 ($y = 1$): split at $0.44 + 0.56 \cdot 0.5 = 0.72$, interval $[0.44, 0.72)$, width $0.28 = 0.7 \cdot 0.8 \cdot 0.5$.

Part II

Building a Context-Mixing Compressor

Chapter 4

Binary Decomposition and Contexts

4.1 Why predict one bit at a time?

A byte has 256 possible values. One could predict a distribution over all 256, and PPM does exactly that. Context mixing instead decomposes each byte into 8 binary decisions, most significant bit first, and predicts each bit with a *binary* model. Reasons:

1. Combining *binary* predictions has a clean, learnable form (logistic mixing, Chapter 6). Combining 256-way distributions has no equally convenient form.
2. Each model needs only one number per context-and-bit-position, so memory and speed are predictable.
3. The binary arithmetic coder is simple and exact.

The cost: 8 coding steps per byte instead of 1, and each model is consulted 8 times. At 180 μs per byte (Chapter 1) this is affordable.

4.2 The partial-byte context c_0

When predicting bit number k ($k = 0 \ldots 7$) of a byte, the bits already coded for that byte are known. Prefix them with a leading 1 to get the number $c_0 \in [1, 255]$, which uniquely identifies the position in the binary tree of the byte.

Example 4.1 — Coding the letter 'e' (= 0x65 = 01100101_2)

bit index k	value to code	c_0 *before* coding	c_0 after
0	0	1 (= 1_2)	2 (= 10_2)
1	1	2	5 (= 101_2)
2	1	5	11 (= 1011_2)
3	0	11	22
4	0	22	44
5	1	44	89
6	0	89	178
7	1	178	357 (= 256 + 101)

After the 8th bit, $c_0 = 357$; subtracting 256 gives the byte value 101 = `0x65` = 'e'. A context model keeps a *separate* probability for each pair (context, c_0): up to 255 probabilities per

context, but in practice only the few nodes on paths actually taken ever get touched.

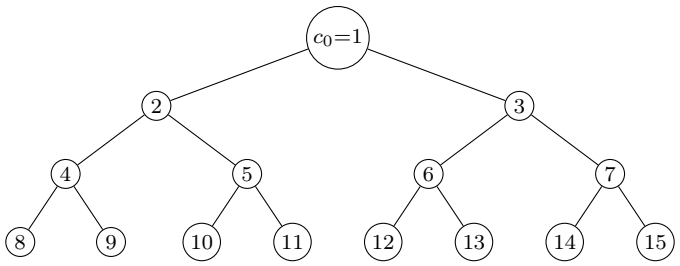


Figure 4.1: The first three levels of the binary tree over byte values. Left edge = bit 0, right edge = bit 1; c_0 of a child is $2c_0$ or $2c_0 + 1$.

4.3 What is a context?

A *context* is any function of the past (and possibly of side information) that can be hashed to an index. The model remembers what happened *every previous time* the same context value occurred. Good contexts partition the past into classes within which the next bit is similar.

Example 4.2 — Contexts for predicting 'a' in "…on the m"

Suppose we have seen "the cat sat on the mat. the cat sat on the m" and are about to code the next byte ('a'):

model	context value	why useful
order 1	`m`	after 'm' vowels are likely
order 2	`␣m`	word-initial 'm'
order 3	`e␣m`	"(the) m"
order 4	`he␣m`	more specific, seen once before
order 6	`␣the m`	seen once before, followed by 'a'
word	hash("m") + previous word "the"	word-level statistics
sparse	bytes 2 and 3 back, skipping 1	robust to small differences
match	long match with the earlier "on the m"	predicts 'a' very confidently

Low orders give general, robust statistics; high orders give precise but rarely-available statistics. No single order is best. That is the motivation for mixing.

Families of contexts used in PAQ-style compressors.

- **Order-n**: the last n bytes ($n = 1..6$ typical, plus a few high orders like 8, 12, 16, 24).
- **Word contexts**: hash of the current partial word, of previous word(s), word bigrams; letter-case folded.
- **Sparse / skip**: some of the last bytes, not contiguous. Useful for binary data and some text structure.
- **Column / record**: position within a fixed-width record (tables, images).
- **Indirect**: the context is the *history of what followed* an earlier context (Section 5.4).
- **Structure-aware (XML / markup)**: for `enwik9`: the enclosing tag, whether we are inside `[[ ]]`, a table, a template (Chapter 10).

4.4 PPM versus context mixing

PPM picks one context, the longest one that has been seen, and uses an "escape" probability to fall back to shorter contexts when the symbol is new. It discards information from all other contexts. Context mixing consults *all* orders simultaneously and lets the data decide, bit by bit, how much to trust each. On text, CM beats PPM by a large margin; in the early prize years the PPM-like `ppmd` variants were substantially behind `paq8`.

4.5 Hashing and memory layout

There are far more possible contexts than memory slots, so contexts are hashed into tables. Typical design (from lpaq/paq8):

- At each *nibble* boundary (every 4 bits) compute a hash of (context hash, partial byte so far) and find the bucket of 16 or 64 bytes holding the states for the 15 or so tree nodes of the next nibble. This costs about *two cache misses per byte per model.*
- Each bucket carries a small checksum so that a collision (a different context mapping to the same bucket) is detected, rather than silently sharing statistics. On a mismatch the bucket is replaced (typically the lower-priority of two or more candidates, judged by the confidence of its first state).
- Tables are sized as powers of two, and for `enwik9` the memory budget (about 10 GB in total) is divided among models.

Example 4.3 — Memory arithmetic

A model on order-6 contexts over 10^9 bytes can see up to 10^9 distinct contexts. At 2 buckets per byte it makes $2 \cdot 10^9$ bucket visits. If you give it 1 GB in 16-byte buckets you have only $6.7 \cdot 10^7$ buckets, so *most* contexts must be evicted: the replacement policy decides what the model remembers. This is why careful tuning of table sizes and replacement rules gives real gains at `enwik9` scale, and why gains measured on `enwik8` may not carry over.

Hash collisions without checks

The teaching compressor in Appendix A omits checksums for clarity. With a 2^{22}-slot table and a 1 MB input collisions are rare; with 10^9 bytes they would be devastating. Never use unchecked hashing at scale.

Chapter 5

Predicting from One Context: Counters and Bit Histories

5.1 Plain frequency counts and the KT estimator

The simplest predictor of the next bit in a context keeps two counts n_0, n_1. The *Krichevsky–Trofimov (KT)* estimator is

$$\hat{p}(1) = \frac{n_1 + \frac{1}{2}}{n_0 + n_1 + 1}.$$

It gives 0.5 with no data, 0.75 after one 1, 0.833 after two 1's, 0.875 after three. It never reaches 0 or 1, avoiding catastrophic costs. (The constant $\frac{1}{2}$ is the optimal additive smoothing under a minimax criterion for a stationary Bernoulli source.)

5.2 Adaptive-rate probability counters

Storing two counts costs memory. A more compact alternative stores a probability p and a small count n, updated by

$$p \leftarrow p + \frac{y - p}{n + 1.5}, \qquad n \leftarrow \min(n + 1,\ \text{limit}).$$

This is the "StateMap" slot in Appendix A (22-bit p and 10-bit n in one 32-bit word).

Proposition 5.1. With no limit on n and $p_0 = \frac{1}{2}$, the update gives exactly $p_n = \dfrac{n_1 + \frac{1}{4}}{n + \frac{1}{2}}$, where n_1 is the number of 1s among the first n bits.

Proof by induction. For $n = 0$: $p_0 = \frac{0.25}{0.5} = \frac{1}{2}$. If $p_n(n+\frac{1}{2}) = n_1 + \frac{1}{4}$ then $p_{n+1} = \dfrac{p_n(n + \frac{1}{2}) + y}{n + \frac{3}{2}} = \dfrac{n_1 + y + \frac{1}{4}}{(n+1) + \frac{1}{2}}$. $\square$

So before the limit is reached the counter is a Dirichlet($\frac{1}{4}$) estimator (slightly more aggressive than KT), and afterwards it becomes an exponential moving average with rate $\approx 1/(\text{limit} + 1.5)$. **The limit controls the stationarity assumption**: a high limit means "this context is stationary, average everything"; a low limit means "recent bits matter more".

Example 5.1 — Stationary versus adaptive

A context sees eight 0s followed by four 1s. The probability that the *next* bit is 1 after each bit:

bit seen:	0	0	0	0	0	0	0	0	1	1	1	1
KT (counts, before update)	.500	.250	.167	.125	.100	.083	.071	.062	.056	.150	.227	.292
rate $1/(n{+}1.5)$, no limit	.167	.100	.071	.056	.045	.038	.033	.029	.132	.214	.283	.340
same, limit $n \le 4$	.167	.100	.071	.056	.045	.037	.030	.025	.202	.347	.466	.563

(The first row gives the prediction made *before* each bit is seen; the other two rows show p *after* the update.) After four 1s the stationary counter says 0.34, while the fast-adapting counter says 0.56: if the source really switched, the latter is closer to the truth; if the 1s were a random fluctuation the former is.

5.3 Bit histories and non-stationary state machines

Real data are *non-stationary*: in a given context the distribution changes over time. Counts throw away *order*. The sequences 000000 11111 and 010101 01010 have similar counts but very different implications.

PAQ replaces (p, n) by a small **state** (one byte) which summarises the history: roughly "n_0 zeros, n_1 ones, and the most recent bit", with the constraint that counts are bounded and that when a bit is observed the *opposite* count is reduced if it is large, so that the state forgets old evidence quickly when the data change. (The exact transition table in PAQ was generated by a program; there are about 255 states.) A simplified rule in the same spirit, illustrative only:

$$\text{observe } y\text{: } n_y \leftarrow n_y + 1;\ \text{if } n_{1-y} > 2 \text{ then } n_{1-y} \leftarrow \lfloor n_{1-y}/2 \rfloor + 1.$$

Starting from $(n_0, n_1) = (8, 0)$ and observing four 1s the state moves $(5,1) \to (3,2) \to (2,3) \to (2,4)$. After three 1s it already says "more 1s than 0s", in agreement with the fast-adapting counter above and unlike the stationary counters.

5.4 Indirect modelling: learning what a history means

The key insight of PAQ: do not convert the state to a probability with a fixed formula. Instead keep a **table mapping each state to a probability that is itself learned**, one table per model (an "adaptive probability map" or *StateMap*):

$$\text{context} \xrightarrow{\text{hash table}} \text{state} \xrightarrow{\text{StateMap}} p \xrightarrow{\text{stretch}} \text{mixer input}.$$

The StateMap is trained on the *outcomes*: whenever a bit y is coded in a context whose state is s, the entry for s is nudged toward y.

Example 5.2 — What the map learns

Consider the state "three 1s, no 0s" in an order-6 context of English text. KT would say $\hat{p} = 0.875$. But in high-order contexts that have been completely deterministic so far, the true frequency of continued determinism is typically much higher (say 0.97), while in an order-1 context the same state means little (perhaps 0.80). Because each model has its *own* StateMap, each learns the right number for *its* situation, and the mixer weight handles the rest. (The numbers 0.97 and 0.80 are illustrative of the effect, not measurements.)

Why indirect beats direct

A direct probability counter must learn each context's probability from scratch: after a single observation it has no idea whether a one-off event is a coincidence or a rule. An indirect model learns, once, across *all* contexts of that model, what a one-off (or a streak) typically implies, and applies it immediately to new contexts. This is statistical borrowing of strength.

The "indirect context model". A different, also called *indirect*, technique: for each order-n context store the *last few bytes that followed it* (e.g. the last two), then use that byte history as a context for a second-level model. It learns patterns like "after this kind of pair the next byte is usually the same as the one before".

5.5 Beyond stretch(p): richer inputs per model

PAQ8 gives the mixer more than one input per context:

- stretch(p): the log-odds (the main input).
- $(p - \frac{1}{2})$ scaled: a *linear-domain* input that gives the mixer a way to express different confidence shapes.
- Special inputs for *deterministic* contexts (only one bit value ever seen) and for *run* information (the last byte seen in this context and how many times in a row).
- A zero input for contexts that have never been seen: stretch$(0.5) = 0$, so an unseen context simply *abstains* from the vote.

Mini-experiment

Take the code in Appendix A and replace each model's slot with a direct frequency counter (two 16-bit counts, KT estimate). Measure the size change on text. Then add a second input per model with $(p - 0.5)$. This is exactly the kind of ablation that will tell you how much each idea is worth on *your* data.

Exercise 5.1 — Counter limits

Start from $p = 0.5$, $n = 0$ and apply the update $p \leftarrow p + (y - p)/(n + 1.5)$ with limit 2 (so $n \leq 2$) to the bit sequence $1, 1, 1, 1, 0$. *Answer:* $p = 0.8333$ (rate $1/1.5$), 0.9 (rate $1/2.5$), 0.9286 (rate $1/3.5$); from here n stays frozen at 2, so the rate stays $1/3.5 = 0.2857$: $p = 0.9490$ after the fourth 1, and $p = 0.9490 - 0.2857 \cdot 0.9490 = 0.6779$ after the final 0.

Chapter 6

The Heart of Context Mixing: Logistic Mixing

6.1 The problem

At each bit we have N models, each giving a probability $p_i = P_i(y = 1)$. We need *one* probability p. We want a rule that (a) is cheap, (b) can express "trust model 3 a lot when the context is XML", (c) can be *trained online* as the data stream by, and (d) combines evidence sensibly.

6.2 Stretch and squash

Definition 6.1. The *logit* or *stretch* function and its inverse, the *logistic* or *squash* function:

$$\text{stretch}(p) = \ln \frac{p}{1-p}, \qquad \text{squash}(x) = \frac{1}{1+e^{-x}}, \qquad \text{squash}(\text{stretch}(p)) = p.$$

stretch(p) is the log-odds of the event. It maps probabilities $(0, 1)$ to the whole real line, with stretch($\frac{1}{2}$) $= 0$, positive when 1 is more likely, negative when 0 is, and symmetric: stretch($1-p$) $=$ $-$ stretch(p).

p	stretch(p)	fixed-point value $\lfloor 256 \text{ stretch}(p) \rceil$ (as in the code)
0.001	-6.907	-1768
0.01	-4.595	-1176
0.1	-2.197	-562
0.25	-1.099	-281
0.5	0	0
0.75	1.099	281
0.9	2.197	562
0.99	4.595	1176
0.999	6.907	1768

Table 6.1: Stretch values. The integer code clamps the domain to ± 2047 ($= \pm 8$ in real units), i.e. $p \in [0.00034, 0.99966]$.

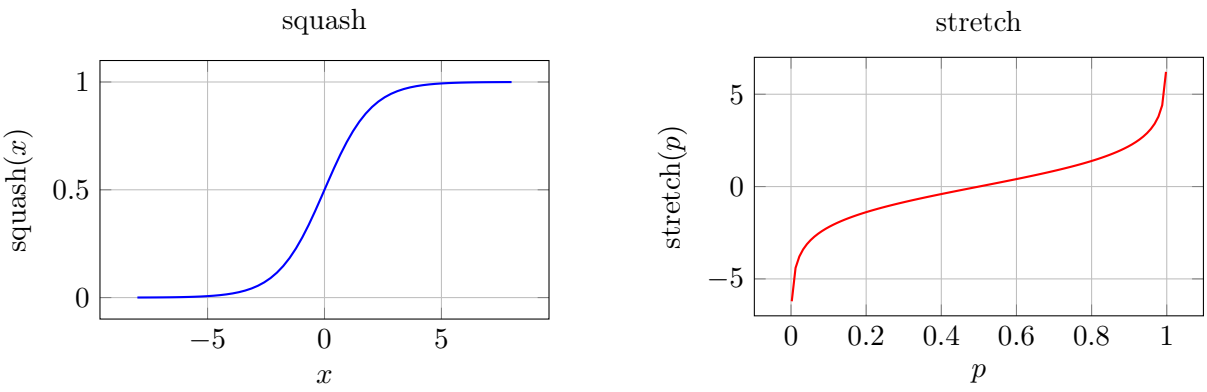


Figure 6.1: Squash and stretch. Stretch magnifies the region near 0 and 1 where the *costly* mistakes live.

6.3 Logistic mixing

Definition 6.2 (Logistic mixing)**.** Given inputs $s_i = \text{stretch}(p_i)$ and weights w_i (one weight vector per *weight set*, chosen by a small context), the mixed probability is

$$\boxed{p = \text{squash}\left(\sum_{i=1}^{N} w_i s_i\right)}$$

This is a single artificial neuron with a sigmoid output (logistic regression). Its training signal is the very quantity we want to minimise, the coding cost.

6.3.1 Why in the logistic domain? The product-of-experts view

Define the *geometric* mixture of the experts' distributions: $P(y) \propto \prod_i P_i(y)^{w_i}$. For a binary outcome,

$$\ln \frac{P(1)}{P(0)} = \sum_i w_i \ln \frac{P_i(1)}{P_i(0)} = \sum_i w_i s_i,$$

so $P(1) = \text{squash}(\sum w_i s_i)$. Logistic mixing is exactly the (normalised) weighted geometric mean of the models' distributions, a *product of experts*. Contrast with the *linear* mixture $p = \sum_i \lambda_i p_i$.

Example 6.1 — Two confident, independent models

Two models each predict $P(1) = 0.99$.

- **Linear mixing** with $\lambda = (\frac{1}{2}, \frac{1}{2})$: $p = 0.99$. Two agreeing experts give no extra confidence.
- **Logistic mixing** with $w = (1, 1)$: $\text{squash}(2 \cdot 4.595) = \text{squash}(9.19) = 0.9999$. Cost if the bit is 1: 0.00014 bits instead of 0.0145.

If the two models see *independent* evidence (different contexts that both say "1"), adding log-odds is Bayes' rule (naive Bayes): the evidence accumulates. The learned weights w_i then correct for the fact that real models are *not* independent (weights below 1) or that some models are more reliable (weights above 1). Linear mixing can never produce a probability more extreme than its most extreme input, which is a severe limitation for highly predictable data like text.

Example 6.2 — Disagreement

Model A says 0.9, model B says 0.1 with equal weights $w = (0.5, 0.5)$. Then $s = (2.197, -2.197)$ and $p = \text{squash}(0) = 0.5$: a sensible "don't know". With $w = (0.8, 0.3)$ the sum is $0.8 \cdot 2.197 - 0.3 \cdot 2.197 = 1.10$ and $p = 0.75$: trust A more. Weights may also be *negative*: a model that is systematically wrong in some context gets a negative weight and its vote is inverted. Linear mixing with non-negative weights cannot do this.

6.4 Learning the weights online

Let the loss be the coding cost of the bit, $\ell = -\ln P(y) = -y \ln p - (1-y)\ln(1-p)$ (natural log; the base only scales the learning rate), with $p = \text{squash}(z)$, $z = \sum_i w_i s_i$.

Theorem 6.1 (Gradient of the coding cost)**.** $\dfrac{\partial \ell}{\partial w_i} = -(y-p)\, s_i$.

Proof. $\frac{d\,\text{squash}}{dz} = p(1-p)$, and $\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{p-y}{p(1-p)}$. By the chain rule $\frac{\partial \ell}{\partial z} = p - y$, and $\frac{\partial z}{\partial w_i} = s_i$. □

So **online gradient descent** is simply

$$\boxed{w_i \leftarrow w_i + \eta\,(y-p)\,s_i}$$

where η is the learning rate. Read it aloud: "increase the weight of every model that voted in the direction of the actual bit, in proportion to how strongly it voted and how wrong the mixture was."

Two lucky facts

(1) Because the loss is the log-loss and the output is a sigmoid, the awkward factor $p(1-p)$ cancels. The gradient does not vanish when the mixer is confidently wrong. (With squared-error loss it would; this is one reason PAQ is trained with log-loss.) (2) The loss is *convex* in w, so there are no local minima and online gradient descent enjoys regret guarantees of order $\sqrt{T}$ against the best fixed weight vector in hindsight.

6.5 A complete numerical example

Example 6.3 — One mixing step, by hand

Three models predict $P(1) = 0.9,\ 0.6,\ 0.3$. Current weights $w = (0.4, 0.3, 0.3)$, learning rate $\eta = 0.02$.

Forward. $s = (\text{stretch}(0.9), \text{stretch}(0.6), \text{stretch}(0.3)) = (2.1972,\ 0.4055,\ -0.8473)$. The dot product is

$$\begin{aligned} z &= 0.4 \cdot 2.1972 + 0.3 \cdot 0.4055 + 0.3 \cdot (-0.8473) \\ &= 0.8789 + 0.1216 - 0.2542 = 0.7463, \end{aligned} \qquad p = \text{squash}(0.7463) = 0.6784.$$

(A plain weighted average of the probabilities would have given $0.4 \cdot 0.9 + 0.3 \cdot 0.6 + 0.3 \cdot 0.3 = 0.63$.)

Case A: the bit is $y = 1$. Cost before: $-\log_2 0.6784 = 0.5598$ bits. Error $y - p = 0.3216$. Weight updates $\Delta w_i = 0.02 \cdot 0.3216 \cdot s_i$:

$$\Delta w = (+0.0141,\ +0.0026,\ -0.0055), \qquad w' = (0.4141,\ 0.3026,\ 0.2945).$$

Model 1 (which strongly said "1") gains weight; model 3 (which said "0") loses it. Recomputing with the new weights gives $p' = 0.6863$, cost 0.5430 bits: after *one* update the same inputs would have been coded 0.017 bits cheaper.
Case B: the bit is $y = 0$. Cost before: $-\log_2(1 - 0.6784) = 1.6366$ bits. Error $y - p = -0.6784$, $\Delta w = (-0.0298, -0.0055, +0.0115)$, $w' = (0.3702, 0.2945, 0.3115)$, new $p' = 0.6612$, cost 1.5617 bits. Model 1 is penalised for having been confidently wrong.

6.6 Fixed-point implementation

The compressor in Appendix A is integer-only (deterministic and fast):

- inputs $s_i = \text{stretch}(p_i) \in [-2047, 2047]$ (real value $\times 256$);
- weights w_i in 16.16 fixed point (65536 = 1.0), initialised to $2^{14} = 0.25$;
- output $p = \text{squash}\big((\sum_i s_i w_i) \gg 16\big)$, a 12-bit probability;
- `err = ((y<<12) - p) * lr`, then `w[i] += (s[i]*err + 0x8000) >> 16`.

Working through the scalings (inputs $\times 256$, weights $\times 2^{16}$, error $\times 4096 \cdot lr$, final shift 2^{-16}) the integer update equals the floating-point rule $w_i \leftarrow w_i + \eta(y - p)s_i$ with

$$\eta = \frac{lr}{4096}.$$

So the value $lr = 6$ used in Appendix A corresponds to $\eta \approx 0.0015$; practical integer rates lie roughly in 2–16, i.e. $\eta \approx 0.0005$–0.004. (The learning rate 0.02 in the hand example above was chosen large so that the weight changes are visible at four decimals.)

The learning rate is the most sensitive parameter

Too small: the mixer cannot follow changes in the data. Too large: the weights jitter and the code length rises (the ablation in Appendix A shows that disabling learning altogether costs 17%). Good compressors use a *decaying* or *adaptive* rate, different per mixer, tuned by experiment; some modern variants replace plain SGD by RMS-normalised or Adam-style updates.

6.7 Weight sets selected by context

A single weight vector must be right everywhere. Instead keep *many* weight vectors and use a small context m to choose which one to use and train:

$$p = \text{squash}\Big(\sum_i w_{m,i} s_i\Big).$$

The selector m can be the partial byte c_0 (the best model mix differs between the first bit of a byte, which is nearly deterministic for ASCII text, and the last), the length of the current match, the previous byte, the bit position, the current XML state… The trade-off is dilution: with M sets each trains on $1/M$ of the data. Typical designs use several mixers with *different* selectors and combine them (next section).

6.8 Two-layer mixing networks

PAQ8 uses a small neural network of two layers:

1. **Layer 1:** K mixers, each using the same N inputs but a different selector context m_k. Each outputs $p^{(k)} = \text{squash}(\sum_i w_{m_k,i} s_i)$.
2. **Layer 2:** a final mixer takes $\text{stretch}(p^{(k)})$, $k = 1..K$ as inputs, with its own weight set (selected by another small context), and produces the final probability.

Each layer-1 mixer is trained to minimise the coding cost *on its own* output; the final mixer is trained on the final output.

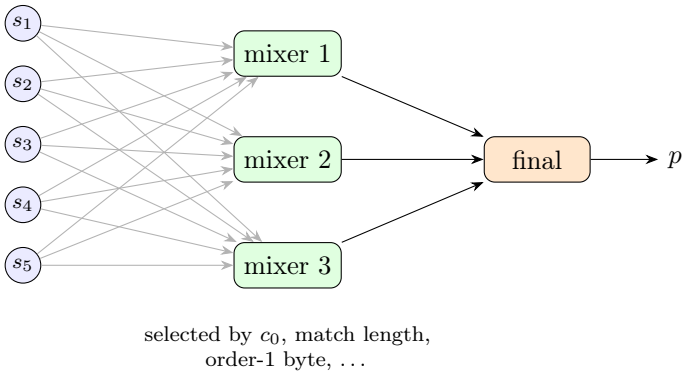


Figure 6.2: A two-layer mixer: several mixers with different weight-set selectors, then a small final mixer.

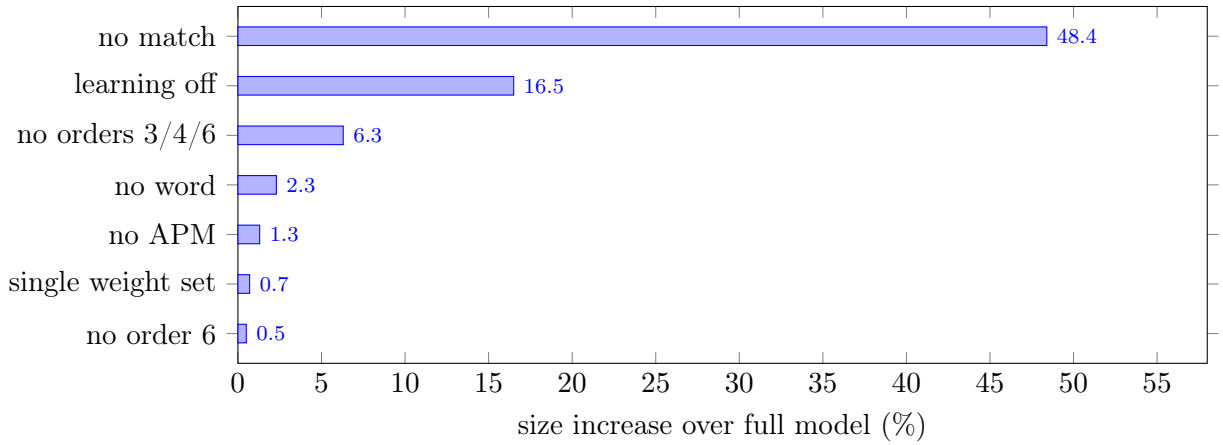
Why does it help? Each first-layer mixer is a *specialist* for one way of slicing the data; the second layer learns which specialist to trust at the moment. It is an ensemble of ensembles. In practice adding a second layer and a handful of well-chosen selectors gives a few percent.

6.9 Ablation results from the reference implementation

I ran the compressor of Appendix A, switching off one component at a time, on a 1 MB English text sample (strongly repetitive license/documentation text; see the Preface for the caveat).

configuration	size (bytes)	vs. full
full model (orders 0,1,2,3,4,6 + word + match + 2 APMs)	89 778	–
no word model	91 847	+2.3%
single weight set (instead of 256 selected by c_0)	90 404	+0.7%
no APM stages	90 925	+1.3%
no order-6 context	90 190	+0.5%
no orders 3, 4, 6	95 424	+6.3%
mixer learning disabled (fixed weights 0.25)	104 626	+16.5%
no match model	133 226	+48%

Table 6.2: Ablation on a 1 MB sample. The enormous value of the match model is a property of *this* sample (many near-duplicate documents); on Wikipedia it is far smaller, though still essential.



Exercise 6.1 — Reproduce the example

Compute by hand the mixed probability for inputs $p = (0.8, 0.7)$ with weights $w = (0.6, 0.6)$. *Answer:* stretch $= (1.3863, 0.8473)$; $z = 0.6 \cdot 2.2336 = 1.3402$; $p = 0.7926$. Note this is *higher* than either input would suggest under averaging (0.75), because the two models agree.

Exercise 6.2 — Gradient check

In the example above with $y = 1$ show that the cost decreases for a small step in the negative gradient direction. *Hint:* the first-order change in cost is $-\eta \|(y - p)s\|^2 < 0$.

Chapter 7

SSE / APM: Fixing Calibration After Mixing

7.1 Idea

The mixer's output can be systematically miscalibrated: perhaps, in some situation, when it says 0.62 the bit is 1 only 55% of the time, or 75%. A **secondary symbol estimation** (SSE) or **adaptive probability map** (APM) stage learns a *correction function* $p \mapsto p'$ *for each small context*. It is a learned calibration curve.

Implementation (as in Appendix A): for each context m keep 33 values $t_{m,0..32}$, the corrected probabilities at the 33 stretch values $-8, -7.875, \ldots, +8$ (spacing 128 in fixed-point). To refine a probability p:

1. compute $s = \text{stretch}(p)$ in fixed-point, locate it between two neighbouring nodes $j, j+1$ with interpolation weight $\omega = (s - s_j)/128$;
2. output $p' = (1-\omega)\, t_{m,j} + \omega\, t_{m,j+1}$;
3. after coding bit y, move the *nearer* node toward y: $t \leftarrow t + (y\text{-target} - t)/2^r$ (rate $r = 7$).

Nodes are initialised to the *identity* map $t_{m,j} = \text{squash}((j-16)\cdot 128)$, so at the beginning the APM changes nothing.

Example 7.1 — One APM lookup and update

The mixer outputs $p = 0.62$. In fixed-point $s = \lfloor 256\,\text{stretch}(0.62) \rceil = 125$. Then $(s + 2048)\cdot 32 = 69\,536$; the node index is $69\,536 \gg 12 = 16$ and the interpolation weight is $69\,536 - 16\cdot 4096 = 4000$ out of 4096 ($= 0.977$), i.e. the input lies 97.7% of the way from node 16 ($s = 0$) to node 17 ($s = 128$). With the identity initialisation $t_{16} = 2048\cdot 16$ and $t_{17} = 2549\cdot 16$ (units: 12-bit probability $\times 16$):

$$p' = \frac{32768\cdot 96 + 40784\cdot 4000}{2^{16}} = 2537 \;\Rightarrow\; \frac{2537}{4096} = 0.619.$$

Now the bit is $y = 1$. The nearer node is 17 (index $16 + (4000 \gg 11) = 17$). The update target is $g = 2^{16} + 2^7 - 2 = 65662$ and $t_{17} \leftarrow 40784 + \big((65662 - 40784) \gg 7\big) = 40784 + 194 = 40978$. The same input now yields $p' = 2549/4096 = 0.6223$. If, in this context, mixer outputs near 0.62 really are followed by 1 about 75% of the time, the node values will drift upward over time and the APM will output ≈ 0.75.

7.2 Chaining and averaging

PAQ-style final stages typically use two or more APMs with *different* contexts (e.g. c_0; the previous byte and c_0; a hash of the order-2 context; the match state) and then average the outputs with the original, e.g. $p_{final} = \frac{1}{4}(2p_{mix} + p_1 + p_2)$ as in Appendix A. The average is a safety measure: an APM with a sparse context is noisy and averaging reduces its variance. In the ablation, removing both APMs costs 1.3%.

Key idea

The mixer is trained to be good *on average* over its weight-set selector; the APM corrects the residual calibration error *conditionally on additional context*, including contexts the mixer never saw. SSE therefore adds information, not just calibration.

Chapter 8

The Match Model and Long-Range Structure

8.1 Why a separate model for long repeats

Context models of order up to about 6 are dense. A context of order 20 or 32 is *almost always novel* unless a long string repeats, and if it repeats, the next symbol is very likely to repeat too. The **match model** handles this cheaply: instead of a hash table of statistics per context, it keeps one pointer into the history.

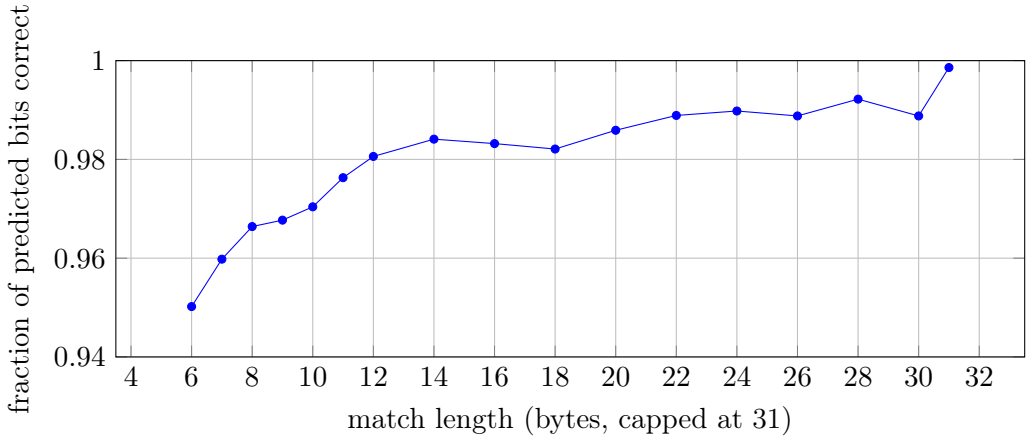
8.2 Algorithm

```
after each byte c is coded and appended to buf[0..pos):
    if (len > 0 and buf[ptr] == c)  { len++;  ptr++ }       // keep following
    else len = 0
    h = hash(last MINLEN bytes)
    if (len == 0) {
        ptr = table[h]                                       // most recent occurrence
        if (ptr > 0) while (len < MAXLEN and buf[ptr-len-1] == buf[pos-len-1]) len++
    }
    table[h] = pos

for each bit of the next byte:
    if (len > 0 and the bits coded so far agree with buf[ptr]):
        expected = next bit of buf[ptr]
        input = stretch( StateMap[min(len,31)][expected] )   // learned confidence
    else input = 0, len = 0 for this byte
```

Two details deserve emphasis.

The confidence is learned, not hard-coded. The StateMap indexed by (match length bucket, expected bit) learns how reliable a match of a given length is. In my test the learned reliabilities, measured as the fraction of predicted bits that were correct, were:



The curve rises smoothly (a match of 6 bytes is right 95% of the time, but a match of 31 or more is right 99.86% of the time). *This is a property of my highly repetitive test file*; on Wikipedia the curve is lower and flatter, and you should measure it yourself. The point is that the mixer receives stretch(0.9986) = 6.6 when the match is long and a much smaller number for short matches, so the mixer is effectively told how much to trust the match.

A mismatch cancels the match for the rest of the byte. Once a predicted bit is wrong, the pointer is dropped (and re-found at the next byte boundary through the hash). Production implementations add *recovery*: they remember the old pointer for a few bytes in case the text diverges briefly (a typo, a changed number) and continues the same.

Example 8.1 — A match in action

History: `the quick brown fox jumps over the lazy dog. the quick brown␣`, and suppose the text then continues with an 'x' ("…brown x…") instead of the 'f' of the first occurrence. After the second "`brown␣`" the hash of the last six bytes (`brown␣`) finds the earlier occurrence, and the backward comparison extends the match to the 16 bytes "`the quick brown␣`" (it stops at the beginning of the history). The predicted next byte is 'f' (= `0x66` = 01100110_2), the actual byte is 'x' (= `0x78` = 01111000_2). Bits 0–2 $(0, 1, 1)$ agree with the prediction and the match model votes confidently (reliability for length 16). At bit 3 the prediction (0) is wrong; the cost is the usual large penalty for a confident miss, the pointer is dropped for the rest of the byte, and the match is re-acquired through the hash after the byte ends.

8.3 Making the match model smarter

- **Several match models** with different minimum lengths (e.g. 4, 8, 20, 32), each with its own pointer: the short one finds recent-but-weak matches, the long one finds reliable ones.
- **Verify the candidate**: store a few candidate positions per hash and choose the longest actual match.
- **Expected byte as a context**: use the predicted byte (and the match length bucket) as the context of an ordinary context model and as the selector for a mixer weight set. That lets the system learn: "when the match predicts a vowel it is right, when it predicts a digit it often fails".
- **Use it in the APM**: SSE indexed by (expected bit, length bucket).

8.4 Long-range structure in Wikipedia

The XML dump contains massive structural repetition: the per-article page skeleton (page, title, id and revision tags), contributor lines, boilerplate in templates, repeated link text, tables and lists in similar formats. A match model sees this immediately, while order-n models for $n \leq 6$ see only fragments. This is also where *article ordering* matters, because the match model prefers related text to be close: see Chapter 10.

Chapter 9

Putting It Together: One Bit, Start to Finish

9.1 The full loop

```
for each byte:
    for each bit position:
        for each model i:   p_i = lookup(context_i, c0)            // state -> StateMap ->
    prob
                            s_i = stretch(p_i) (plus extra inputs)
        for each mixer k:   z_k = sum_i w[k][sel_k][i] * s_i ; p_k = squash(z_k)
        p_mix = final_mixer(stretch(p_k))
        p = average( p_mix, APM_1(p_mix, ctx_1), APM_2(p_mix, ctx_2) )
        encode(bit, p)                                             // or decode
        --- both sides now know the bit y ---
        for each model:     update state/counter for the bit
                            update the StateMap entry toward y
        for each mixer:     w += eta * (y - p_k) * s_i
        update APMs, c0 = 2*c0 + y
    at byte boundary: update histories, hashes, match model, word hash, ...
```

9.2 A numerical walk-through of a single bit

Example 9.1 — Predicting the third bit of 'e' after "th"

We are coding 'e' (01100101_2) in the context "...in th". We are at bit index 2 with $c_0 = 5$ (bits 01 already coded). The next bit is $y = 1$. Models report $P(y = 1)$:

input	source	p_i	$s_i = \text{stretch}(p_i)$	weight w_i (set c_0=5)	cost if y=1 (bits)
1	order 0	0.55	0.2007	0.10	0.862
2	order 1 ('h')	0.80	1.3863	0.20	0.322
3	order 2 ("th")	0.93	2.5867	0.30	0.105
4	order 3 ('␣th')	0.97	3.4761	0.25	0.044
5	word model	0.90	2.1972	0.15	0.152
6	match (inactive)	–	0	0.00	–
7	bias (constant 1)	–	1	−0.05	–

Mixer: $z = 0.10 \cdot 0.2007 + 0.20 \cdot 1.3863 + 0.30 \cdot 2.5867 + 0.25 \cdot 3.4761 + 0.15 \cdot 2.1972 - 0.05 \cdot 1 =$

2.2219, so $p_{mix} = \text{squash}(2.2219) = 0.9022$. (The best single model alone would cost 0.044 bits, the mixture costs 0.1485 bits *because the weights are not yet trained to trust order 3 more*; this is just a snapshot.)
APM stage: suppose APM_1 (context c_0) returns 0.915 and APM_2 (context previous byte + c_0) returns 0.931 (hypothetical values). Final probability: $\frac{1}{4}(2 \cdot 0.9022 + 0.915 + 0.931) = 0.9126$.
Coding: the bit is 1: cost $-\log_2 0.9126 = 0.1319$ bits. The coder narrows its interval by the factor 0.9126.
Learning (with $\eta = 0.02$): the error uses the *mixer's own* output: $y - p_{mix} = 0.0978$. Weight changes $\Delta w_i = \eta \cdot 0.0978 \cdot s_i = (0.0004, 0.0027, 0.0051, 0.0068, 0.0043, 0, 0.0020)$, new weights $(0.1004, 0.2027, 0.3051, 0.2568, 0.1543, 0, -0.0480)$; with these the same inputs would give $p_{mix} = 0.9067$. The counters and StateMaps of each model are also moved toward $y = 1$, the APM nodes nearest to the inputs move toward 1, and $c_0 \leftarrow 11$.

9.3 What dominates the time?

- **Memory latency.** Each model costs about two random cache misses per byte (one per nibble), $\approx 100\,\text{ns}$ each. With 40 models, $\approx 8\,\mu\text{s}$ per byte just waiting for memory unless you *prefetch* (compute next-nibble hashes early and issue prefetch instructions).
- **Mixer arithmetic.** N inputs $\times$ K mixers $\times$ (dot product + update) per bit; with $N \approx 500$, $K \approx 10$ this is $\approx 10^4$ multiply-adds per bit, perfectly suited to SIMD (16-bit integer inputs and weights with `pmaddwd`-style instructions).
- **Neural components.** An LSTM with hidden size H costs $\sim 8H(H + I)$ multiply-adds per step for the forward pass alone (Chapter 11), vastly more than everything else if H is in the hundreds.

Exercise 9.1 — Extend the walk-through

Suppose the match model became active with expected bit 1 and a learned reliability of 0.9986, with weight 0.4. Recompute z and p_{mix}. *Answer:* $s = \text{stretch}(0.9986) = 6.570$, $z = 2.2219 + 0.4 \cdot 6.57 = 4.850$, $p_{mix} = 0.9922$.

Part III

Toward the Hutter Prize

Chapter 10

Modelling Wikipedia: Text, XML and Preprocessing

10.1 What is in `enwik9`?

`enwik9` is the first 10^9 bytes of the XML dump of the English Wikipedia taken on 3 March 2006. It is not clean prose. Typical content:

- XML skeleton per article: page, title, id and revision elements, timestamp, contributor, comment, and the text element carrying the article body.
- MediaWiki markup inside the text: double-bracket links (with optional labels), double-brace templates, bold and italic quote marks, equals-sign headings, tables, lists, categories, redirects.
- XML/HTML character entities (for quotes, ampersands, angle brackets), because the dump is XML-escaped.
- English prose, numbers and dates, names, and a lot of non-English words and UTF-8 multibyte sequences.

Good compression requires a model for *each* of these "sub-languages" and a mixer that knows which one it is currently in. Obtain the file yourself from Mahoney's site (it is about 300 MB zipped) and *look at it* with a hex viewer and `less` before designing anything: a few hours reading the data pays for itself.

10.2 Preprocessing: transform the data before modelling

All top Hutter Prize entries since the early years use a reversible **preprocessing transform** that rewrites the text into a form that is easier to model; the decompressor undoes it. Typical transforms:

Capital conversion. Replace an uppercase letter by a flag byte followed by the lowercase letter (or flag a whole capitalised word). "The" and "the" now share statistics.

Dictionary (word-replacement) coding. Replace common words and phrases by short codes (one or two bytes). The text shrinks and the contexts become *more informative per byte*: an order-3 context now spans maybe 3 words instead of 3 letters. This is typically a net win of several percent, and the top entries use large, carefully constructed dictionaries.

XML / markup tokenisation. Replace tags and common markup by single symbols.

Number and date normalisation, entity decoding, UTF-8 handling.

The size of the dictionary counts

Because S includes the decompressor, a dictionary of D bytes must save more than D bytes of compressed output (and the dictionary itself can be stored compressed). A word that occurs once in `enwik9` is never worth putting in the dictionary; a word that occurs a million times certainly is. The optimum is found by experiment, and the transformation is a major tuning area. The preprocessor associated with early Hutter Prize record holders (the "HP-2017" preprocessor named in public STARLIT repository documentation) is specific to Wikipedia text.

Example 10.1 — A toy dictionary transform

Original: `The United States of America was...` Step 1 (capital flag ↑ = next letter was uppercase): `↑the ↑united ↑states of ↑america was...` Step 2 (dictionary codes #n): ↑#1 ↑#2 ↑#3 #4 ↑#5 #6. The 36-character phrase becomes about 12 symbols, and the model's *order-2* context now covers the equivalent of 2 words. The decoder inverts the steps in reverse order.

10.3 Article reordering

The articles in `enwik9` appear in a fixed order unrelated to topic similarity. Compression improves when related articles are *adjacent*, because the match model and the high-order contexts (and the LSTM state) can reuse recent vocabulary and phrasing. The STARLIT submission (2021) exploited exactly this: according to its repository it permutes the articles of `enwik9` before compressing with `cmix`, and stores the permutation so the decompressor can restore the original order; the cost of storing the permutation is part of S. This is a good illustration of the general principle: **any side information you add must pay for itself**.

10.4 Specialised context models for Wikipedia

On top of the generic orders 1–6 and high orders, text-oriented CM compressors add:

- **Word model**: hash of the current word prefix; of the previous word and current prefix; of the two previous words (bigram/trigram); letter-case folded. Capture "next word given previous words".
- **Sparse and skip contexts**: e.g. bytes 2–3 back; useful when a small variation breaks the contiguous context.
- **Structure state**: inside a tag name? inside an attribute value? inside `[[ ]]`, `{{ }}`, a table row? depth of nesting. The *state* is used both as a context (hashed with the order-n context) and as the *mixer selector*, so the weights specialise to the structure.
- **Link / title echo**: the `<title>` of an article appears again in its first sentence and links; a model can predict the repeated string from the title (a match model restricted to the title is a cheap way).
- **Numeric models**: digits following a pattern (years $19xx$, consecutive ids, timestamps incrementing).

- **UTF-8 position**: for multibyte sequences, continuation bytes are almost deterministic given the lead byte.
- **Indirect contexts and run models** (Chapter 5) at several orders.

Find where the bits go

Instrument your compressor to accumulate $-\log_2 p$ *per category* of the input: XML tags, markup, English words, numbers, non-ASCII, whitespace. This breakdown tells you where the remaining bytes are. Experienced Hutter Prize participants spend a large fraction of their time on such diagnostics; a model improvement that fixes a category that accounts for 1% of the output cannot yield more than 1%.

10.5 Bytes, words or tokens?

An LLM-style tokenizer (BPE) is, from this point of view, just another dictionary transform; whether BPE-like tokens help a CM depends on whether the gain in context span beats the loss of byte-level sharing between related forms ("walk", "walks", "walked"). The literature of the prize suggests a hybrid: dictionary-code the very common words, leave the rest as letters.

10.6 Tuning on `enwik8` versus `enwik9`

`enwik8` (the first 10^8 bytes) is a convenient development set, and `enwik9` is the real target. Differences arise: more history makes high-order and match models stronger; the hash tables are under far greater pressure; slowly-adapting parameters (counter limits, learning-rate decay) have a different optimum. Budget time to re-tune on the full file. The prize FAQ explicitly allows (and indeed encourages) tailoring a compressor to `enwik9`.

Chapter 11

Neural Components: the LSTM and `cmix`

11.1 cmix in one paragraph

`cmix` (Byron Knoll) is an open-source compressor in the PAQ tradition that combines a very large number of models (PAQ8-derived context models, many specialised models, and neural networks, notably LSTM byte predictors) into a layered mixing network followed by APM/SSE stages. It is very slow (hundreds of times slower than `lpaq`) and achieves among the best published results on text. The recent `enwik9` records (`fx-cmix`, `fx2-cmix`; Table 1.1) are tuned derivatives of the `cmix` line. *I describe the architecture in general terms here; you must read the* ***`cmix`*** *source on GitHub to learn the actual configuration, which changes between versions.*

11.2 The LSTM predictor

A recurrent network reads the byte stream one symbol at a time and keeps a hidden state that summarises *everything it considers relevant* about the past, including long-range and semantic information that no n-gram context can see. The long short-term memory (LSTM) cell, with input x_t, previous hidden state h_{t-1} and cell state c_{t-1}:

$$
\begin{aligned}
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) && \text{forget gate}\\
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) && \text{input gate}\\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) && \text{output gate}\\
\tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) && \text{candidate}\\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cell update}\\
h_t &= o_t \odot \tanh(c_t) && \text{hidden state}
\end{aligned}
$$

A softmax output layer turns h_t into a probability distribution $q(\cdot)$ over the 256 possible next bytes.

Size and cost. With hidden size H and input size I (e.g. a one-hot byte, $I = 256$), one layer has $4H(H+I+1)$ parameters and one forward step costs about $4H(H+I)$ multiply-adds, plus the output layer $256H$. For $H = 256$, $I = 256$: $\approx 5.3 \cdot 10^5$ parameters and $\approx 5.3 \cdot 10^5$ multiply-adds (≈ 1 MFLOP) per byte forward, roughly 2–3 times that to train by back-propagation through time.

11.3 From a byte distribution to bit probabilities

The arithmetic coder consumes *bit* probabilities, so a byte-level predictor must be converted. For the bit at tree node c_0, the probability that it is 1 is the mass of byte values consistent with the prefix *and* the bit 1, divided by the mass consistent with the prefix:

$$P(y_k = 1 \mid \text{prefix}) = \frac{\sum_{b \in B_1(\text{prefix})} q(b)}{\sum_{b \in B(\text{prefix})} q(b)}.$$

Example 11.1 — Two-bit toy alphabet

Alphabet $\{00, 01, 10, 11\}$ with network output $q = (0.5, 0.2, 0.2, 0.1)$.

- First bit: $P(1) = q(10) + q(11) = 0.3$.
- Second bit given first bit 0: $P(1 \mid 0) = \dfrac{q(01)}{q(00) + q(01)} = \dfrac{0.2}{0.7} = 0.2857$.
- Second bit given first bit 1: $P(1 \mid 1) = \dfrac{q(11)}{q(10) + q(11)} = \dfrac{0.1}{0.3} = 0.3333$.

Check: the probability of the symbol 10 is $P(\text{first} = 1) \cdot P(\text{second} = 0 \mid 1) = 0.3 \cdot (1 - 0.3333) = 0.2 = q(10)$. The decomposition reproduces the original distribution exactly.

These bit probabilities are then *stretched and fed to the mixer like any other model's prediction*; the mixer learns how much to trust the LSTM, bit by bit and in each selector context. This hybrid is the essential design: the LSTM is an additional expert, complementary to the count-based models.

11.4 Online training

There is no pre-training. The network is trained *while compressing*: after each byte (or every few bytes) the actual byte is the training target and the weights are updated by truncated back-propagation through time over a window of recent bytes, using an optimiser such as Adam or RMSprop-style normalised steps (as far as I know `cmix` uses an Adam-like optimiser; check the source). The decompressor does the identical computation in the identical order, so it re-learns exactly the same weights.

Learned parameters are free; computation is not

Because the LSTM weights are learned on the fly from a fixed initialisation (a seed), they are *not part of S*. A pre-trained network of the same size would have to be shipped and would cost its full size in bytes. The Prize therefore rewards models that learn quickly from a single pass over 1 GB, within the time limit. This is why the model class that wins is "online-learning ensemble" rather than "big pretrained model", and why a general-purpose LLM, however good at English, does not win (the prize FAQ mentions the compute limits; recent work such as *Language Modeling Is Compression* (Delétang et al., 2023) makes the same point about model size counting).

11.5 Budgeting compute for a neural model

Example 11.2 — How much LSTM fits?

Allowed: 50 hours for 10^9 bytes = 180 μs/byte. On one core, suppose a sustained 10–30 GFLOP/s with well-vectorised code (single-core AVX2 peak is higher but memory bound code is slower). Then the *whole* model gets 1.8–5.4 MFLOP per byte. A single-layer LSTM with $H = 256$ needs about 1 MFLOP forward and 2–3 MFLOP backward per byte: it would consume most of the budget by itself, leaving little for the 100+ context models. Hence the real systems use cleverness: smaller hidden sizes, training only every few bytes, sparse or low-precision updates, sharing the cost across the model's inputs. Estimating this budget before writing any code is a critical design step.

11.6 Other neural ideas

- **Bigger or deeper recurrent models** (multi-layer LSTMs, GRUs) and **Transformers** trained online. Fabrice Bellard's NNCP project is, as I recall, the best-known system doing lossless compression with Transformer/LSTM networks and ranks near the top of Mahoney's Large Text Compression Benchmark, but it does not fit the Prize's time limits; check its documentation for current numbers.
- **Neural mixers.** Replace the logistic mixer with a small neural network that also sees context features.
- **Mixing at the byte level**: combine byte-level distributions (from the LSTM) before decomposing to bits.
- **Distilling** expensive models into cheap ones is not allowed to cheat the limits, but *architecture search* on a development subset is.

11.7 Floating-point determinism

Compression and decompression must perform *bit-identical* computations. With integer-only CM this is automatic; with floating-point neural networks you must ensure: the same binary and the same CPU instruction paths on both sides; a fixed order of summation (no multithreaded reductions with non-deterministic order); no dependence on uninitialised memory or on timing; and care with `-ffast-math` and library `exp`/`tanh` implementations. The safest course is to use your own implementations of `exp`, `tanh` and `sigmoid` (polynomial or table based) in fixed code.

Exercise 11.1 — Byte-to-bit decomposition

A network outputs q over 8 byte values (3-bit symbols 000..111): $(0.3, 0.1, 0.1, 0.1, 0.2, 0.1, 0.05, 0.05)$. Compute the three bit probabilities for coding the symbol 101. *Answer:* first bit: $P(1) = 0.2 + 0.1 + 0.05 + 0.05 = 0.4$; given prefix 1, second bit: $P(1 \mid 1) = (0.05 + 0.05)/0.4 = 0.25$; given prefix 10, third bit: $P(1 \mid 10) = 0.1/(0.2 + 0.1) = 0.3333$. The coded bits are $1, 0, 1$, so the symbol's probability is $0.4 \cdot (1 - 0.25) \cdot 0.3333 = 0.1 = q(101)$, as it must be.

Chapter 12

Engineering Under the Prize Limits

12.1 The constraints, restated as design requirements

constraint (check the current rules!)	design consequence
≈ 50 h, single core	$\leq 180\,\mu$s per byte for the whole model; no multithreading
≈ 10 GB RAM	table sizes, bucket formats, LSTM state, history buffer
S = decompressor + archive	code size and dictionary size count
decoder = encoder cost	you pay the time twice when testing
exact reproduction on their machine	determinism; no machine-specific behaviour

12.2 Speed engineering

- **Cache discipline.** Place all the states for one nibble of one context in a single 64-byte cache line. Compute the hashes for the next nibble as soon as the current one is known, and issue `__builtin_prefetch`.
- **SIMD for mixers.** Keep inputs and weights as 16-bit integers; use multiply-add instructions that accumulate pairs into 32 bits (e.g. AVX2 `vpmaddwd`). Align arrays to 32 bytes. Mixer cost then becomes a few cycles per 16 inputs.
- **Avoid division and `exp`** in the inner loop: use lookup tables for stretch and squash.
- **Profile with real counters** (`perf`): memory stalls and branch mispredictions rather than instruction count are usually what limits CM.
- **Two-pass tricks**: the encoder may analyse the file (choose a dictionary, a parameter set, an article order) and transmit the result; the decoder just uses it. Analysis time on the encoder side is not repeated by the decoder, but the total encoder limit still applies.

12.3 Memory engineering

- Count bytes per context slot honestly and give each model a budget; the best split is found empirically.
- Use *checked* buckets with sensible replacement (Chapter 4).
- Huge pages (e.g. 2 MB) reduce TLB misses in multi-gigabyte tables.

- Bound the history buffer; for `enwik9` the whole file (10^9 bytes) can stay in memory for the match model.

12.4 Correctness engineering

A compressor that cannot decompress is worth nothing, and bugs in CM are subtle because the encoder and decoder share almost all code. A disciplined test regime:

1. A **round-trip test on every change**, on a small file (1 MB) and a medium file (100 MB). Compare with `cmp` or `sha1sum`.
2. **Determinism test**: run compression twice and require byte-identical archives. Run decompression on a different (but same-architecture) machine.
3. **Never let encoder-only information leak into the model**: for example, using the *next* byte or the file length while predicting. The decoder cannot do that. A classic way to catch this: structure the code so that a single `predict()/update(bit)` interface is used by both encoder and decoder.
4. **Edge cases**: empty input, 1-byte input, runs of identical bytes, all 256 byte values, files smaller than the hash window.
5. **Sanitisers** (`-fsanitize=address,undefined`) on the small test; uninitialised memory is a common source of non-determinism.

Floating point and compilers

Changing the compiler version or flags can change the results of floating-point code (fused multiply-add contraction, vectorised reductions). If both compressor and decompressor are built from the same source with the same flags on the same machine they will agree; but any rebuild between compress and decompress runs risks catastrophic failure. For the Prize, the decompressor is shipped as a self-extracting executable, which removes the risk at submission time, but *you* must check by running that exact executable.

12.5 Development workflow

1. **Fast loop:** first 10–20 MB of `enwik8` (seconds to minutes). Use for algorithmic ideas; ignore changes below 0.1% unless cheap.
2. **Medium loop:** full `enwik8` or the first 100–200 MB of `enwik9` (hours); run many experiments in parallel on a multi-core development machine. (The *submission* is single-core but your *development* need not be.)
3. **Full loop:** complete `enwik9`, 50 hours; do this rarely and always with a final-candidate build, and keep logs (size, time, peak memory) for each.
4. Log everything: git commit hash, parameters, size to the byte, time, memory.

12.6 Parameter tuning

A CM compressor has hundreds of numeric parameters (counter limits, learning rates, table sizes, selector definitions). Practical approaches:

- **Coordinate descent**: change one parameter at a time up and down by a factor, keep improvements. Slow but robust.
- **Random search / evolutionary** on the medium loop with many cores. The record holders reportedly relied heavily on automated tuning.
- **Bayesian optimisation** for small numbers of expensive parameters.

Beware *noise from interactions*: parameters tuned on 20 MB may be wrong at 1 GB (Chapter 10); re-verify the big ones on the medium loop.

12.7 Shrinking the decompressor

The decompressor binary and its data (dictionaries, initial tables) count. Techniques: compile with `-Os` versus `-O3` experimentally (speed versus size); strip symbols; compress the dictionary with the model itself (bootstrapping: decode the dictionary using a smaller model first); remove unused code paths (the decoder never needs the encoder's analysis code).

Exercise 12.1 — Budget your cache misses

With 80 context models, each needing 2 random DRAM accesses per byte at 90 ns without prefetching, compute the memory time per byte and compare with the 180 μs budget. *Answer:* $80 \cdot 2 \cdot 90$ ns $= 14.4\,\mu$s (8% of the budget); with prefetching most of this overlaps with computation. Memory latency is *not* the main problem at this model count; compute (mixers, neural networks) is.

Chapter 13

Designing Your Own Algorithm

13.1 An honest assessment

To receive any money you must beat the current record by at least 1% ($\approx$ 1.1 MB) and the record has been advanced mostly by large, long-developed systems and by a few people who understand every line of them. It is a legitimate and valuable goal; a realistic plan is staged, with each stage useful in itself (a lecture, a paper, a benchmark entry on Mahoney's *Large Text Compression Benchmark*) even if the final prize is out of reach.

13.2 Staged plan

Stage 1. Reproduce. Understand and run the code of Appendix A; reproduce the ablation table; implement the same ideas yourself from a blank file. Read `lpaq1` completely (about 500 lines).

Stage 2. PAQ8 level. Add: bit-history state machines and StateMaps, nibble-bucketed checked hash tables, two-layer mixers, several APMs, run/indirect/sparse/word models, several match models. Validate against `lpaq`/`paq8` binaries on `enwik8`.

Stage 3. Text specialisation. XML/markup model, capital conversion, dictionary preprocessor, article reordering experiments. Build the per-category bit-cost diagnostics.

Stage 4. Neural component. Add an LSTM (start small), feeding the mixer. Measure gain per millisecond.

Stage 5. Match the state of the art. Run `cmix` (and, if you can, its `fx` variants) on the same data. Until you match them on your development set you have nothing new to contribute.

Stage 6. Engineer for the limits. Profile, vectorise, prefetch; confirm the 50-hour, single-core, 10 GB envelope on a clean machine.

Stage 7. Innovate. Only now explore genuinely new ideas, ranked by (expected gain)/(effort).

13.3 A checklist for evaluating every idea

For each proposed change answer, *in writing*:

1. Which category of the bit-cost breakdown does it target, and how many bytes of output does that category currently account for? (This is the upper bound on the gain.)
2. What does it cost in time per byte and in memory?
3. What does it cost in decompressor size?
4. Does the gain survive at the large scale (medium loop)?
5. Is the gain *additive* with the others? (Run the ablation with and without.)

13.4 Where gains may still be hiding

These are research directions, not promises; every item has probably been tried in some form:

- Better **tokenisation/dictionaries** jointly optimised with the model; learned dictionaries; handling of rare words and names.
- **Better counters/state machines** for `enwik`-style data; two-rate counters; state machines trained from data rather than hand-designed.
- **Mixer architecture**: richer selectors (grammatical state, title match), three layers, neural mixers, better learning-rate schedules.
- **SSE** contexts derived from the LSTM state or the match state.
- **Neural models**: bigger or better-structured recurrent models; attention over the recent history; mixture of small networks specialised by structure.
- **Two-pass modelling**: encoder-side search over parameters and orderings, transmitting a few kilobytes of side information.
- **Precision**: higher-resolution probabilities in the final stages and the coder (Chapter 3).
- **Exploiting the Wikipedia 2006 specifics**: templates, categories and interlanguage links have regularities that dedicated models can predict.

13.5 Social and practical advice

- Join the discussion forum linked from the prize page, and read the threads of previous winners: they often describe the changes that mattered.
- Read the licences: the Prize requires an OSI-approved open-source licence for the winning submission, and building on `cmix`/`paq8` code means complying with their licences.
- Re-read the official rules before you begin, before you spend 50 hours on a run, and before you submit. They determine eligibility, hardware and the 30-day comment period.

Exercise 13.1 — Where can the gain come from?

Suppose your breakdown shows XML tags use 2% of the compressed output, markup syntax 8%, English words 70%, numbers 6%, other 14%. To gain 1% overall, by what percentage must you reduce the English-words category? the XML-tags category? *Answer:* words: $1/70 = 1.43\%$; tags: $1/2 = 50\%$ (implausible). Prioritise the large categories unless the small one is cheap.

Appendix A

A Complete Teaching Compressor (tinycm)

This is a complete, working, readable context-mixing compressor, about 190 lines of C. It is *not* competitive with `paq8` or `cmix`; it exists so that every idea in Part II can be seen in running code and modified. I compiled it with `gcc -O2` and verified that decompression reproduces the input exactly (byte-for-byte `cmp`).

A.1 Architecture

component	implementation
context models	orders 1, 2, 3, 4, 6 and a word model, each hashed into a 2^{22}-slot table; order 0 direct
slot	32 bits: 22-bit probability and 10-bit count, adaptive rate $1/(n+1.5)$, limit 255
match model	hash of last 6 bytes, longest-recent-match pointer; reliability learned per length bucket
mixer	single layer, 9 inputs (stretch of 8 models and a bias), 256 weight sets selected by c_0
APM stages	two: context c_0 and (previous byte, c_0); averaged 2:1:1 with the mixer output
coder	32-bit carry-less binary arithmetic coder, 12-bit probabilities

Omitted on purpose (they are exercises): bit-history states and StateMaps (the slots hold direct probabilities), checksummed buckets, multiple mixers and a second layer, run/indirect/sparse models.

A.2 Results on a 1 MB sample

For orientation: at this speed 10^9 bytes would take about 20 minutes; the Prize allows 50 hours, a factor of 150 more.

A.3 Source code

compressor	size of 1 000 000-byte sample	bits per byte
gzip -9	214 589	1.717
bzip2 -9	134 193	1.074
xz -9e	102 576	0.821
tinycm (this appendix)	**89 778**	**0.718**

Table A.1: The sample is 1 MB of English documentation and licence text taken from the preparation machine (not Wikipedia; highly repetitive). tinycm runs at about 1.4 s per MB for both compression and decompression (≈ 0.7 MB/s).

```c
/* tinycm.c -- a small, readable context-mixing compressor (teaching code).
 * Usage:  tinycm c in out   |   tinycm d in out                          */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <ctype.h>
typedef uint8_t U8; typedef uint32_t U32; typedef uint64_t U64; typedef uint16_t U16;

/* ---------- logistic helpers: squash(x)=4096/(1+e^-x/256), stretch=inverse ---------- */
static int squash(int d) {
  static const int t[33] = {1,2,3,6,10,16,27,45,73,120,194,310,488,747,1101,
    1546,2047,2549,2994,3348,3607,3785,3901,3975,4022,4050,4068,4079,4085,
    4089,4092,4093,4094};
  if (d > 2047) return 4095;
  if (d < -2047) return 0;
  int w = d & 127; d = (d >> 7) + 16;
  return (t[d] * (128 - w) + t[d + 1] * w + 64) >> 7;
}
static short stretch_t[4096];
static void init_stretch(void) {
  int pi = 0;
  for (int x = -2047; x <= 2047; x++) {
    int v = squash(x);
    for (int i = pi; i <= v; i++) stretch_t[i] = x;
    pi = v + 1;
  }
  for (int i = pi; i < 4096; i++) stretch_t[i] = 2047;
}
#define stretch(p) stretch_t[p]

/* ---------- adaptive probability slot: 22-bit probability + 10-bit count ---------- */
static int dt[1024];
static inline int  slot_p(U32 s) { return s >> 20; }            /* 12-bit probability */
static inline void slot_update(U32 *s, int y, int limit) {
  int n = *s & 1023, p = *s >> 10;
  if (n < limit) ++*s; else *s = (*s & 0xfffffc00) | limit;
  int64_t delta = (int64_t)(((y << 22) - p) >> 3) * dt[n];
  *s += (U32)delta & 0xfffffc00;
}

/* ---------- APM / SSE stage: 33 interpolation nodes per context ---------- */
typedef struct { U16 *t; int idx; } APM;
static void apm_init(APM *a, int n) {
  a->t = malloc(n * 33 * sizeof(U16));
  for (int i = 0; i < n; i++)
    for (int j = 0; j < 33; j++) a->t[i * 33 + j] = squash((j - 16) * 128) * 16;
}
static int apm_pp(APM *a, int pr, int cx) {
```

```
  int s = (stretch(pr) + 2048) * 32, lo = s >> 12, wt = s & 4095;
  a->idx = cx * 33 + lo + (wt >> 11);
  return (a->t[cx * 33 + lo] * (4096 - wt) + a->t[cx * 33 + lo + 1] * wt) >> 16;
}
static void apm_update(APM *a, int y, int rate) {
  int g = (y << 16) + (y << rate) - y - y;
  a->t[a->idx] += (g - a->t[a->idx]) >> rate;
}

/* ---------- the predictor ---------- */
#define NCM 6                 /* hashed context models: o1 o2 o3 o4 o6 word  */
#define NIN (NCM + 3)         /* + order-0, match model, bias                */
#define TBITS 22
#define HBITS 22              /* match-model hash table size                 */
#define BUFSZ (1 << 26)
static U32 *T[NCM], o0[256], mm_sm[64 * 2];
static U32 ctx[NCM], idx[NCM], *o0p, *mmp;
static int st[NIN], wsel;
static int *W;                /* weight sets: 256 sets x NIN weights (16.16)  */
static APM a1, a2;
static int c0 = 1, bitpos = 0, pr = 2048, pr_mix = 2048;
static U64 c8 = 0;            /* last 8 bytes                                */
static U32 wordh = 0;
static U8 *buf; static U32 pos = 0, mptr = 0, mlen = 0, *mht;
static int mexp = -1;         /* expected bit from the match model, or -1    */

static void predictor_init(void) {
  init_stretch();
  for (int i = 0; i < 1024; i++) dt[i] = 16384 / (i + i + 3);
  for (int i = 0; i < NCM; i++) {
    T[i] = malloc((size_t)4 << TBITS);
    for (U32 j = 0; j < (1u << TBITS); j++) T[i][j] = 1u << 31;
  }
  for (int i = 0; i < 256; i++) o0[i] = 1u << 31;
  for (int i = 0; i < 128; i++) mm_sm[i] = 1u << 31;
  W = malloc(256 * NIN * sizeof(int));
  for (int i = 0; i < 256 * NIN; i++) W[i] = 1 << 14;      /* 0.25 */
  apm_init(&a1, 256); apm_init(&a2, 65536);
  buf = calloc(BUFSZ, 1); mht = calloc(1u << HBITS, 4);
}
static inline U32 hidx(U32 h, U32 c) {
  U32 x = (h + c * 0x9E3779B1u) * 0x85EBCA6Bu;
  x ^= x >> 15; x *= 0xC2B2AE35u;
  return x >> (32 - TBITS);
}
static int predict(void) {
  for (int i = 0; i < NCM; i++) {
    idx[i] = hidx(ctx[i], c0);
    st[i] = stretch(slot_p(T[i][idx[i]]));
  }
  o0p = &o0[c0]; st[NCM] = stretch(slot_p(*o0p));
  /* match model: only active if the bits seen so far agree with prediction */
  mexp = -1;
  if (mlen > 0 && ((buf[mptr] | 256) >> (8 - bitpos)) == c0) {
    mexp = (buf[mptr] >> (7 - bitpos)) & 1;
    int l = mlen > 31 ? 31 : mlen;
    mmp = &mm_sm[l * 2 + mexp];
    st[NCM + 1] = stretch(slot_p(*mmp));
  } else { mlen = 0; st[NCM + 1] = 0; }
  st[NCM + 2] = 256;                                      /* bias */
  wsel = c0 * NIN;
  int64_t dot = 0;
  for (int i = 0; i < NIN; i++) dot += (int64_t)st[i] * W[wsel + i];
```

```
  pr_mix = squash((int)(dot >> 16));
  int p1 = apm_pp(&a1, pr_mix, c0);
  int p2 = apm_pp(&a2, pr_mix, (c0 | (int)(c8 & 255) << 8) & 0xffff);
  pr = (2 * pr_mix + p1 + p2 + 2) >> 2;
  if (pr < 1) pr = 1;
  if (pr > 4095) pr = 4095;
  return pr;
}
static void update(int y) {
  for (int i = 0; i < NCM; i++) slot_update(&T[i][idx[i]], y, 255);
  slot_update(o0p, y, 255);
  if (mexp >= 0) slot_update(mmp, y, 1023);
  int err = ((y << 12) - pr_mix) * 6;                          /* mixer learning rate */
  for (int i = 0; i < NIN; i++) W[wsel + i] += (st[i] * err + 0x8000) >> 16;
  apm_update(&a1, y, 7); apm_update(&a2, y, 7);
  c0 = c0 * 2 + y; bitpos++;
  if (mexp >= 0 && mexp != y) mlen = 0;
  if (bitpos == 8) {                                           /* byte finished */
    int c = c0 & 255;
    buf[pos++ & (BUFSZ - 1)] = c;
    c8 = (c8 << 8) | c;
    wordh = isalpha(c) ? (wordh + tolower(c) + 1) * 0x2F0B4A27u : 0;
    static const U64 mask[5] = {0xff, 0xffff, 0xffffff, 0xffffffffULL, 0xffffffffffffULL};
    for (int i = 0; i < 5; i++) ctx[i] = (U32)(((c8 & mask[i]) * 0x9E3779B97F4A7C15ULL) >>
    32) + i;
    ctx[5] = wordh * 0x3D4D51CBu + 99;
    /* match model: follow current match or look up the last 6 bytes */
    if (mlen > 0 && buf[mptr] == c) { mlen++; mptr++; } else mlen = 0;
    U32 h = (U32)(((c8 & 0xffffffffffffULL) * 0x9E3779B97F4A7C15ULL) >> (64 - HBITS));
    if (mlen == 0) {
      mptr = mht[h];
      if (mptr > 0) while (mlen < 255 && mlen < mptr &&
            buf[(mptr - mlen - 1) & (BUFSZ-1)] == buf[(pos - mlen - 1) & (BUFSZ-1)]) mlen++;
    }
    mht[h] = pos;
    c0 = 1; bitpos = 0;
  }
}

/* ---------- binary arithmetic coder (carry-less, 32-bit) ---------- */
static U32 x1 = 0, x2 = 0xffffffff, x = 0;
static FILE *fin, *fout;
static void encode(int y, int p) {
  U32 xmid = x1 + ((x2 - x1) >> 12) * p;
  y ? (x2 = xmid) : (x1 = xmid + 1);
  while (((x1 ^ x2) & 0xff000000) == 0) { putc(x2 >> 24, fout); x1 <<= 8; x2 = (x2 << 8) |
    255; }
}
static int decode(int p) {
  U32 xmid = x1 + ((x2 - x1) >> 12) * p;
  int y = x <= xmid;
  y ? (x2 = xmid) : (x1 = xmid + 1);
  while (((x1 ^ x2) & 0xff000000) == 0) {
    x1 <<= 8; x2 = (x2 << 8) | 255; x = (x << 8) | (getc(fin) & 255);
  }
  return y;
}
int main(int argc, char **argv) {
  if (argc != 4) { fprintf(stderr, "usage: tinycm c|d in out\n"); return 1; }
  fin = fopen(argv[2], "rb"); fout = fopen(argv[3], "wb");
  if (!fin || !fout) { perror("open"); return 1; }
  predictor_init();
  if (argv[1][0] == 'c') {
```

```
    fseek(fin, 0, SEEK_END); U32 n = ftell(fin); fseek(fin, 0, SEEK_SET);
    fwrite(&n, 4, 1, fout);
    for (U32 i = 0; i < n; i++) {
      int c = getc(fin);
      for (int b = 7; b >= 0; b--) { int y = (c >> b) & 1; encode(y, predict()); update(y); }
    }
    for (int i = 0; i < 4; i++) { putc(x1 >> 24, fout); x1 <<= 8; }  /* flush */
  } else {
    U32 n; fread(&n, 4, 1, fin);
    for (int i = 0; i < 4; i++) x = (x << 8) | (getc(fin) & 255);
    for (U32 i = 0; i < n; i++) {
      int c = 0;
      for (int b = 0; b < 8; b++) { int y = decode(predict()); update(y); c = c * 2 + y; }
      putc(c, fout);
    }
  }
  fclose(fout); return 0;
}
```

A.4 Suggested modifications (a research-training programme)

1. **Measure.** Reproduce the ablation table of Chapter 6 (Table 6.2) on your own data. Add a per-component bit-cost log.
2. Replace slots by **bit-history states** + **StateMap** (Chapter 5) and measure.
3. Add **checksummed buckets** and test with a file larger than the table can hold.
4. Add **two more mixers** with selectors (match length; previous byte) and a **final mixer** (Chapter 6.8).
5. Add a second input per model, $(p - \frac{1}{2})$, and confidence inputs.
6. Add a **sparse** model and a **second match model** with minimum length 12.
7. Replace the hash $\rightarrow$ slot design by one slot per *nibble* (Chapter 4.5).
8. Add a small **LSTM** in floating point and feed its bit-probability to the mixer.
9. Make mixer learning rates **decay**; add RMS-normalised updates.
10. Write the **pre-processing** transform of Chapter 10 and measure on a Wikipedia sample.

Appendix B

Notation

symbol	meaning
$y,\ y_t$	the bit being coded (0 or 1)
c_0	partial-byte context: leading 1 followed by the bits of the current byte seen so far
p_i	probability that $y = 1$ predicted by model i
$\mathrm{stretch}(p) = \ln \frac{p}{1-p}$	logit; in the code scaled by 256 and clamped to ± 2047
$\mathrm{squash}(x) = 1/(1 + e^{-x})$	logistic function (inverse of stretch)
$s_i = \mathrm{stretch}(p_i)$	mixer input
$w_{m,i}$	mixer weight for input i in weight set m
η	mixer learning rate
ℓ	code length, $-\log_2$ of the probability assigned to what happened
$H,\ D_{\mathrm{KL}}$	entropy, Kullback–Leibler divergence
$S,\ L,\ Z$	Prize total size, previous record, prize fund

Appendix C

Glossary

APM / SSE adaptive probability map / secondary symbol estimation: a learned calibration of a probability, conditioned on a small context.

Arithmetic coding entropy coding that turns a sequence of probabilities into a number inside nested intervals.

Bit history / state a one-byte summary of the sequence of bits seen in a context.

Context any function of the past used to index statistics.

Context mixing (CM) combining the predictions of many context models with an adaptively trained function.

Indirect model a model whose context is the history of what followed an earlier context, or which maps a bit history to a probability through a learned table.

Logistic mixing $p = \text{squash}(\sum w_i s_i)$; a product of experts.

Match model predicts the next symbol by following the most recent earlier occurrence of the current long context.

Online learning parameters updated while the data stream by; no separate training phase.

StateMap table mapping a bit-history state (or context) to an adaptively learned probability.

Stretch / squash logit and logistic functions.

Weight set one of many weight vectors of a mixer, selected by a context.

Appendix D

Suggested Reading and Code to Study

Read the source code in this order.

1. `lpaq1` (Matt Mahoney): about 500 lines, the cleanest complete CM compressor; compare every function with this document.
2. `paq8` family (`paq8l`, and the actively developed `paq8px`): the full model zoo.
3. `cmix` (Byron Knoll): the PAQ8 models plus LSTM and the layered mixer.
4. The `starlit` repository (Margaritov): the article-reordering approach and how the prize constraints were met.

Bibliography

[1] C. E. Shannon, "A mathematical theory of communication", *Bell System Technical Journal* 27 (1948).

[2] I. H. Witten, R. M. Neal, J. G. Cleary, "Arithmetic coding for data compression", *Communications of the ACM* 30(6), 1987.

[3] J. G. Cleary and I. H. Witten, "Data compression using adaptive coding and partial string matching", *IEEE Trans. Communications* 32(4), 1984.

[4] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, Wiley.

[5] M. Mahoney, *Data Compression Explained*, online book, http://mattmahoney.net/dc/dce.html.

[6] M. Mahoney, "Adaptive weighing of context models for lossless data compression", Florida Tech. technical report CS-2005-16.

[7] M. Mahoney, *Large Text Compression Benchmark*, http://mattmahoney.net/dc/text.html.

[8] M. Hutter et al., *500 000€ Prize for Compressing Human Knowledge* (rules, FAQ, record table), http://prize.hutter1.net/.

[9] M. Hutter, *Universal Artificial Intelligence*, Springer, 2005.

[10] G. E. Hinton, "Training products of experts by minimizing contrastive divergence", *Neural Computation* 14(8), 2002.

[11] S. Hochreiter and J. Schmidhuber, "Long short-term memory", *Neural Computation* 9(8), 1997.

[12] D. P. Kingma and J. Ba, "Adam: a method for stochastic optimization", ICLR 2015.

[13] J. Duda, "Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding", arXiv:1311.2540, 2013.

[14] G. Delétang et al., "Language modeling is compression", arXiv:2309.10668 (ICLR 2024).

[15] B. Knoll, *cmix*, https://github.com/byronknoll/cmix.

[16] F. Bellard, *NNCP: lossless data compression with neural networks*, https://bellard.org/nncp/.

[17] A. Margaritov, *STARLIT*, https://github.com/amargaritov/starlit.